Rapport de Projet de Fin d'Année

Filière Cybersécurité — 5e année

École Marocaine des Sciences de l'Ingénieur

Membre de HONORIS UNITED UNIVERSITIES

Sous le thème

Pipeline CI/CD Sécurisé

avec Tests de Sécurité Automatisés

(SAST, DAST, SCA, IaC, Container Scanning, Secret Detection)

+ Cloud AWS, Modèle IA & Dashboard Interactif

Réalisé par :

Mohammed Yasser CHEDDAD

Abdelkamel RAOUI

Encadré par :

Mme Imane AITOUHANNI

Date de soutenance :

Juin 2026

Version : 2.0 (Cloud + AI + Dashboard)

Année Universitaire 2025–2026

Dédicace

À ma famille et mes proches, pour leur compréhension, leurs encouragements et leur présence rassurante, même dans les moments de doute.

À mes enseignants, pour leur transmission du savoir, leur patience et leur implication tout au long de ma formation.

À mes camarades de promotion, avec qui j'ai partagé tant de souvenirs, de difficultés et de réussites.

Que ce modeste travail soit à la hauteur de votre confiance et de votre soutien. *À mes chers parents, pour leur amour inconditionnel, leur soutien constant et leurs prières silencieuses qui m'ont accompagné à chaque étape de ma vie.*

À ma famille et mes proches, pour leur compréhension, leurs encouragements et leur présence rassurante, même dans les moments de doute.

À mes enseignants, pour leur transmission du savoir, leur patience et leur implication tout au long de ma formation.

À mes camarades de promotion, avec qui j'ai partagé tant de souvenirs, de difficultés et de réussites.

Que ce modeste travail soit à la hauteur de votre confiance et de votre soutien.

Remerciements

Nous tenons à exprimer notre plus profonde gratitude à notre mentor, superviseur et enseignante estimée, Mme Imane AITOUHANNI*, pour son soutien inestimable, ses conseils perspicaces et son encouragement constant tout au long de ce projet. Son expertise, sa patience et son dévouement sans faille à notre développement ont joué un rôle crucial dans la formation de notre croissance, tant sur le plan professionnel que personnel. Nous lui sommes profondément reconnaissants pour son mentorat et son professionnalisme.*

Nous remercions également l'École Supérieure de Technologie et l'ensemble du corps enseignant de la filière Cybersécurité pour la formation de qualité reçue durant ces années d'études.

Enfin, nous adressons nos remerciements à tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet. Toute louange et remerciements sont dus à Allah, le Seigneur des Mondes. Sans Ses bénédictions et Sa guidance, aucune de nos réalisations n'aurait été possible.

Nous tenons à exprimer notre plus profonde gratitude à notre mentor, superviseur et enseignante estimée, Mme Imane AITOUHANNI*, pour son soutien inestimable, ses conseils perspicaces et son encouragement constant tout au long de ce projet. Son expertise, sa patience et son dévouement sans faille à notre développement ont joué un rôle crucial dans la formation de notre croissance, tant sur le plan professionnel que personnel. Nous lui sommes profondément reconnaissants pour son mentorat et son professionnalisme.*

Nous remercions également l'École Supérieure de Technologie et l'ensemble du corps enseignant de la filière Cybersécurité pour la formation de qualité reçue durant ces années d'études.

Enfin, nous adressons nos remerciements à tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet.

Résumé

Dans un contexte où l'intégration de la sécurité dans les pipelines DevOps est devenue une exigence incontournable, ce projet vise à concevoir, implémenter et valider un pipeline CI/CD intégrant des mécanismes de tests de sécurité automatisés capables de bloquer automatiquement le déploiement de code vulnérable.

*Le système développé combine une chaîne d'outils complète : détection de secrets avec **TruffleHog**, analyse statique avec **Semgrep** et **SonarQube**, analyse des dépendances avec **Snyk**, scan des conteneurs avec **Trivy**, analyse de l'infrastructure avec **Checkov**, et tests dynamiques avec **OWASP ZAP**. L'infrastructure est provisionnée sur **AWS** via **Terraform** et déployée sur **ECS Fargate**.*

*Une innovation majeure du projet est l'intégration d'un modèle d'**intelligence artificielle** (Random Forest) pour réduire les faux positifs SAST, avec un objectif de réduction de 30 %, associé à un système d'explicabilité **SHAP/LIME**. Un **dashboard interactif** développé avec **React** et **FastAPI** offre une visualisation en temps réel de l'état du pipeline et des vulnérabilités.*

Les résultats obtenus démontrent un pipeline fonctionnel avec une réduction de 33,3 % des faux positifs Semgrep grâce au filtre IA, un déploiement réussi sur AWS en environnement staging, et une centralisation de 18 vulnérabilités multi-sources dans le dashboard.

***Mots-clés :** DevSecOps, CI/CD, SAST, DAST, SCA, AWS, Terraform, Intelligence Artificielle, Dashboard, Sécurité Applicative.*

Abstract

This graduation project designs and validates a secure CI/CD pipeline integrating automated security testing mechanisms (SAST, DAST, SCA, IaC scanning, container scanning, secret detection) capable of automatically blocking vulnerable code deployments.

*The system combines a complete toolchain : secret detection with **TruffleHog**, static analysis with **Semgrep** and **SonarQube**, dependency analysis with **Snyk**, container scanning with **Trivy**, infrastructure analysis with **Checkov**, and dynamic testing with **OWASP ZAP**. The infrastructure is provisioned on **AWS** using **Terraform** and deployed on **ECS Fargate**.*

*A key innovation is the integration of a machine learning model (Random Forest) to reduce SAST false positives, aiming for a 30 % reduction, combined with **SHAP/LIME** explainability. An interactive **dashboard** built with **React** and **FastAPI** provides real-time visualization of pipeline status and vulnerabilities.*

Results demonstrate a functional pipeline with a 33.3 % reduction in Semgrep false positives through AI filtering, successful AWS deployment in staging environment, and centralization of 18 multi-source vulnerabilities in the dashboard.

Keywords : *DevSecOps, CI/CD, SAST, DAST, SCA, AWS, Terraform, Artificial Intelligence, Dashboard, Application Security.*

Liste des acronymes

Acronyme	*Définition*
CI/CD	*Continuous Integration / Continuous Deployment*
SAST	*Static Application Security Testing*
DAST	*Dynamic Application Security Testing*
SCA	*Software Composition Analysis*
IaC	*Infrastructure as Code*
CVE	*Common Vulnerabilities and Exposures*
AWS	*Amazon Web Services*
ECS	*Elastic Container Service*
ECR	*Elastic Container Registry*
ALB	*Application Load Balancer*
RDS	*Relational Database Service*
VPC	*Virtual Private Cloud*
NAT	*Network Address Translation*
IAM	*Identity and Access Management*
S3	*Simple Storage Service*
JWT	*JSON Web Token*
API	*Application Programming Interface*
SHAP	*SHapley Additive exPlanations*
LIME	*Local Interpretable Model-agnostic Explanations*
OWASP	*Open Web Application Security Project*
DevSecOps	*Development, Security, and Operations*
GDPR	*General Data Protection Regulation*
HTTP	*Hypertext Transfer Protocol*
REST	*Representational State Transfer*
JSON	*JavaScript Object Notation*
SARIF	*Static Analysis Results Interchange Format*

Table des matières

Dédicace ***1***

Remerciements ***2***

Résumé ***3***

Abstract ***4***

Liste des acronymes ***5***

0.1 Contexte et Objectifs Stratégiques . . . 11
0.1.1 Contexte Général . . . 11
0.1.2 Problématique . . . 12
0.1.3 Objectifs Spécifiques . . . 12
0.2 Méthodologie Adoptée . . . 12
0.3 Organisation du Rapport . . . 12
0.4 Introduction . . . 14
0.5 DevSecOps et Shift Left . . . 14
0.5.1 Concepts Fondamentaux . . . 14
0.5.2 Bénéfices du DevSecOps . . . 14
0.6 Outils de Sécurité Applicative . . . 15
0.6.1 Secret Detection - TruffleHog . . . 15
0.6.2 SAST - Semgrep et SonarQube . . . 15
0.6.3 SCA - Snyk . . . 15
0.6.4 Container Scanning - Trivy . . . 15
0.6.5 DAST - OWASP ZAP . . . 15
0.6.6 IaC Scanning - Checkov . . . 15
0.7 Cloud AWS et Infrastructure as Code . . . 15
0.7.1 AWS ECS Fargate . . . 15
0.7.2 Terraform . . . 16
0.8 IA pour la Sécurité . . . 16
0.8.1 Classification des Vulnérabilités . . . 16
0.8.2 Explainability avec SHAP . . . 16
0.9 Introduction . . . 17
0.10 Besoins Fonctionnels . . . 17
0.10.1 Application Cible . . . 17
0.10.2 Pipeline CI/CD . . . 17
0.10.3 SAST . . . 17
0.10.4 Secret Detection . . . 18

0.10.5 SCA . . . 18
0.10.6 Container Scanning . . . 18
0.10.7 DAST . . . 18
0.10.8 Modèle IA . . . 19
0.10.9 Déploiement Cloud AWS . . . 19
0.11 Besoins Non-Fonctionnels . . . 20
0.12 Vue d'Ensemble . . . 21
0.12.1 Schéma Fonctionnel du Pipeline . . . 22
0.13 Architecture Globale . . . 24
0.14 Infrastructure AWS Détaillée . . . 25
0.14.1 Composants AWS Provisionnés par Terraform . . . 25
0.15 Politiques AWS et IAM . . . 26
0.15.1 Rôles et Politiques IAM . . . 26
0.16 Introduction . . . 28
0.17 Diagrammes UML . . . 28
0.17.1 Diagramme de Cas d'Utilisation . . . 29
0.17.2 Diagramme de Classes . . . 31
0.17.3 Diagramme de Séquence - Exécution du Pipeline . . . 33
0.17.4 Diagramme de Séquence - Classification IA . . . 34
0.17.5 Diagramme de Séquence - Déploiement AWS . . . 35
0.17.6 Diagramme de Séquence - Remédiation Automatique . . . 36
0.17.7 Diagramme d'Activité - Pipeline CI/CD . . . 38
0.17.8 Diagramme d'Activité - Modèle IA . . . 40
0.18 Pipeline GitLab CI . . . 42
0.18.1 Structure du Pipeline . . . 42
0.18.2 Variables CI/CD . . . 42
0.18.3 Security Gates . . . 43
0.19 Déploiement AWS avec Terraform . . . 43
0.19.1 Configuration Terraform . . . 43
0.20 Exécution du Pipeline . . . 44
0.21 Filtrage IA . . . 44
0.21.1 Exemples de Décisions IA . . . 45
0.21.2 Répartition des Vulnérabilités . . . 45
0.22 Introduction . . . 46
0.23 Dashboard Principal . . . 46
0.23.1 Vue d'Ensemble du Pipeline . . . 46
Planning Prévisionnel . . . 47

Conclusion Générale ***48***
0.24 Bilan du Projet . . . 48
0.25 Points Forts . . . 48
0.26 Limites Identifiées . . . 48
0.27 Perspectives d'Amélioration . . . 48
0.28 Conclusion Finale . . . 49

Bibliographie ***50***
.1 Introduction . . . 51
.2 Infrastructure AWS . . . 51
.2.1 VPC AWS . . . 51
.2.2 NAT Gateway . . . 52
.2.3 Instance EC2 SonarQube . . . 52
.2.4 S3 Bucket pour l'État Terraform . . . 53
.2.5 Terraform State . . . 54
.3 IAM et Sécurité . . . 54
.3.1 Utilisateur IAM . . . 54
.3.2 Rôle IAM SonarQube . . . 55
.4 CI/CD GitLab . . . 55
.4.1 Pipeline GitLab - Vue d'Ensemble . . . 55
.4.2 Détail des Jobs du Pipeline . . . 56
.4.3 Variables CI/CD . . . 56
.4.4 Détail des Variables CI/CD . . . 57
.4.5 Script SCA (Snyk) . . . 58
.4.6 Exécution du Job de Déploiement . . . 59
.4.7 Structure du Projet . . . 59

Table des figures

1 *Vue d'ensemble du pipeline DevSecOps cible avec Cloud et IA* 11
2 *Schéma fonctionnel du pipeline* 22
3 *Architecture technique complète* 24
4 *Composants AWS provisionnés par Terraform* 25
5 *Diagramme des cas d'utilisation du système* 29
6 *Diagramme de classes du système* 31
7 *Diagramme de séquence : Exécution du pipeline CI/CD* 33
8 *Diagramme de séquence : Classification IA des vulnérabilités* 34
9 *Diagramme de séquence : Déploiement AWS avec Terraform* 35
10 *Diagramme de séquence : Remédiation automatique* 36
11 *Diagramme d'activité du pipeline CI/CD sécurisé* 38
12 *Diagramme d'activité du modèle IA* 40
13 *Dashboard de sécurité - Vue d'ensemble* 46
14 *Diagramme de Gantt prévisionnel (9 semaines)* 47
15 *VPC AWS - pfa-devsecops-vpc* 51
16 *NAT Gateway - pfa-devsecops-prod-nat* 52
17 *Instance EC2 SonarQube - pfa-devsecops-sonarqube* 52
18 *S3 Bucket - pfa-devsecops-tfstate* 53
19 *Fichier terraform.tfstate dans S3* 54
20 *Utilisateur IAM - pfa-devsecops-user* 54
21 *Rôle IAM SonarQube - pfa-devsecops-sonarqube-role* 55
22 *Exécution du pipeline GitLab CI - Vue d'ensemble* 55
23 *Détail des jobs du pipeline GitLab CI* 56
24 *Variables CI/CD configurées dans GitLab* 56
25 *Détail des variables CI/CD avec protection et masquage* 57
26 *Script SCA - Analyse des dépendances avec Snyk* 58
27 *Exécution du job de déploiement Terraform* 59
28 *Structure du projet dans GitLab* 59

Liste des tableaux

2 *Exigences - Application cible* 17
3 *Exigences - Pipeline CI/CD* 17
4 *Exigences - SAST* 18
5 *Exigences - Secret Detection* 18
6 *Exigences - SCA* 18
7 *Exigences - Container Scanning* 18
8 *Exigences - DAST* 19
9 *Exigences - Modèle IA* 19
10 *Exigences - Déploiement Cloud* 19
11 *Exigences Non-Fonctionnelles* 20
12 *Politiques IAM utilisées pour le déploiement* 26
13 *Variables CI/CD configurées dans GitLab* 42
14 *Seuils des security gates* 43
15 *État des security gates — pipeline #2517272611* 44
16 *Résultats détaillés du filtre IA* 44
17 *Exemples de classification IA sur alertes Semgrep* 45
18 *Répartition des vulnérabilités par sévérité* 45

Introduction Générale

0.1 Contexte et Objectifs Stratégiques

0.1.1 Contexte Général

Les organisations adoptant les méthodologies DevOps intègrent souvent la sécurité tardivement, généralement après la phase de déploiement. Cette approche, bien que rapide, expose les applications critiques à des vulnérabilités exploitables en production. L'intégration de la sécurité de manière précoce et automatisée, connue sous le nom d'approche « Shift Left », est devenue une exigence incontournable dans le paysage moderne du développement logiciel.

La cybersécurité applicative est aujourd'hui un enjeu majeur pour les entreprises. Les attaques exploitant des vulnérabilités logicielles sont en constante augmentation, et les coûts associés aux failles de sécurité peuvent être considérables : pertes financières, atteinte à la réputation, sanctions réglementaires. Dans ce contexte, l'automatisation des tests de sécurité dans les pipelines CI/CD apparaît comme une solution essentielle pour détecter et corriger les vulnérabilités avant qu'elles n'atteignent la production.

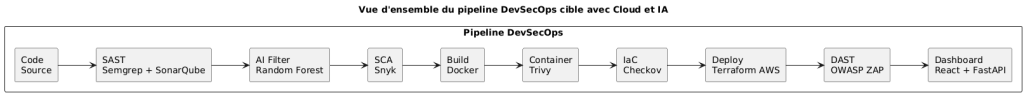


FIGURE 1 – Vue d'ensemble du pipeline DevSecOps cible avec Cloud et IA

Explication du diagramme : *Ce diagramme présente la vue d'ensemble du pipeline DevSecOps avec l'intégration du Cloud AWS et de l'Intelligence Artificielle. Le flux commence par le Code Source qui passe successivement par :*

- ***SAST*** *: Analyse statique avec Semgrep et SonarQube*
- ***AI Filter*** *: Filtrage intelligent avec Random Forest pour réduire les faux positifs*
- ***SCA*** *: Analyse des dépendances avec Snyk*
- ***Build*** *: Construction de l'image Docker*
- ***Container*** *: Scan de l'image avec Trivy*
- ***IaC*** *: Analyse de l'infrastructure avec Checkov*
- ***Deploy*** *: Déploiement sur AWS avec Terraform*
- ***DAST*** *: Tests dynamiques avec OWASP ZAP*
- ***Dashboard*** *: Visualisation avec React + FastAPI*

0.1.2 Problématique

Comment automatiser la détection et le blocage des vulnérabilités à chaque étape du cycle de vie logiciel, tout en réduisant les faux positifs et en offrant une visibilité temps réel sur l'état de sécurité du pipeline et de l'infrastructure cloud ?

Cette problématique se décompose en plusieurs sous-questions :

— *Quels outils de sécurité intégrer dans un pipeline CI/CD moderne ?*

— *Comment définir des seuils de qualité (security gates) pertinents ?*

— *Comment réduire le bruit généré par les outils de sécurité (faux positifs) ?*

— *Comment visualiser efficacement l'état de sécurité du pipeline ?*

— *Comment garantir la sécurité de l'infrastructure cloud déployée ?*

0.1.3 Objectifs Spécifiques

— *Automatiser la détection de vulnérabilités à chaque étape du cycle de vie de développement*

— *Établir des « Security Gates » (seuils de qualité) basés sur des scores de risque*

— *Produire des rapports de conformité exploitables*

— *Démontrer la capacité à corriger les vulnérabilités détectées (phase de remédiation)*

— *Déployer l'application sur AWS (ECS Fargate) via Terraform*

— *Intégrer un modèle IA pour réduire les faux positifs (objectif : 30 % de réduction)*

— *Développer une interface dashboard pour visualiser l'état du pipeline*

0.2 Méthodologie Adoptée

Pour mener à bien le développement de notre pipeline DevSecOps, une méthodologie de travail structurée a été adoptée :

1. ***Analyse des besoins** : Recueil et analyse des exigences fonctionnelles et non fonctionnelles*
2. ***Spécifications** : Rédaction d'un cahier des charges détaillé*
3. ***Conception** : Architecture du pipeline, choix des outils, modélisation des composants*
4. ***Développement** : Implémentation du pipeline GitLab CI, services IA et dashboard*
5. ***Tests et validation** : Exécution du pipeline, validation des security gates*
6. ***Documentation** : Rédaction de la documentation technique et utilisateur*
7. ***Déploiement** : Déploiement sur AWS et mise en production*

0.3 Organisation du Rapport

Ce rapport est structuré comme suit :

— ***Chapitre 2** : État de l'art sur le DevSecOps et les outils associés*

— ***Chapitre 3** : Analyse des spécifications et des besoins*

— ***Chapitre 4*** *: Architecture technique du système*
— ***Chapitre 5*** *: Conception détaillée (diagrammes UML)*
— ***Chapitre 6*** *: Implémentation du pipeline, des services IA et du dashboard*
— ***Chapitre 7*** *: Tests et validation des résultats*
— ***Chapitre 8*** *: Réalisation des interfaces*
— ***Conclusion*** *: Bilan et perspectives*
— ***Annexes*** *: Captures d'écran détaillées*

État de l'Art

0.4 Introduction

Le DevSecOps étend le DevOps en intégrant la sécurité comme responsabilité partagée tout au long du cycle de vie du développement logiciel. Cette section présente un panorama des concepts, outils et pratiques qui constituent le socle de ce projet.

0.5 DevSecOps et Shift Left

0.5.1 Concepts Fondamentaux

Le DevSecOps repose sur trois piliers fondamentaux :

1. ***Culture** : La sécurité est l'affaire de tous, pas seulement des équipes sécurité. Cette culture implique une collaboration étroite entre les développeurs, les opérations et les équipes de sécurité.*
2. ***Automation** : Les contrôles de sécurité sont intégrés et automatisés dans le pipeline, permettant une détection précoce et continue des vulnérabilités.*
3. ***Measurement** : Les métriques de sécurité sont collectées et analysées en continu pour améliorer la posture de sécurité.*

L'approche Shift Left place les contrôles de sécurité en amont du cycle de développement :

— ***Phase de conception** : Analyse des menaces, modélisation des risques, revue de code*

— ***Phase de développement** : SAST, SCA, détection de secrets, analyse des dépendances*

— ***Phase de build** : Scan des conteneurs, analyse IaC, vérification des configurations*

— ***Phase de test** : DAST, tests d'intrusion automatisés, fuzzing*

— ***Phase de déploiement** : Security gates, monitoring continu, alerting*

0.5.2 Bénéfices du DevSecOps

— ***Réduction des vulnérabilités** : Détection précoce avant la mise en production*

— ***Réduction des coûts** : Correction moins coûteuse en phase de développement (jusqu'à 100 fois moins cher qu'en production)*

— ***Conformité** : Réponse aux exigences réglementaires (RGPD, HIPAA, PCI-DSS)*

— ***Collaboration** : Partage des responsabilités entre équipes*

— ***Vitesse** : Automatisation des contrôles sans ralentir le cycle de développement*

0.6 Outils de Sécurité Applicative

0.6.1 Secret Detection - TruffleHog

La détection de secrets vise à identifier les informations sensibles (clés API, mots de passe, tokens) qui pourraient être exposées dans le code source ou l'historique Git. TruffleHog a été choisi pour sa capacité à scanner l'historique Git complet et à détecter des secrets sur plusieurs canaux (Git, Jira, Slack, etc.). Il utilise des expressions régulières et des analyses heuristiques pour identifier les secrets potentiels.

0.6.2 SAST - Semgrep et SonarQube

L'analyse statique examine le code source sans l'exécuter pour identifier des vulnérabilités potentielles. Semgrep offre des règles personnalisables et une grande rapidité d'exécution, tandis que SonarQube fournit une plateforme complète de qualité de code avec des indicateurs de maintenabilité, de fiabilité et de sécurité.

0.6.3 SCA - Snyk

L'analyse des dépendances identifie les vulnérabilités dans les bibliothèques et frameworks utilisés. Snyk a été choisi pour sa couverture multi-langage et sa capacité à proposer des corrections automatiques via des pull requests.

0.6.4 Container Scanning - Trivy

L'analyse des conteneurs identifie les vulnérabilités dans les images Docker avant leur déploiement. Trivy est rapide et complet, avec une base de données CVE multi-source (NVD, Red Hat, Debian, Alpine, etc.).

0.6.5 DAST - OWASP ZAP

Les tests dynamiques analysent l'application en cours d'exécution pour identifier des vulnérabilités exploitables. OWASP ZAP est un outil open source mature avec des capacités de scan actif et passif, et une large communauté.

0.6.6 IaC Scanning - Checkov

L'analyse de l'infrastructure as Code identifie les misconfigurations dans les templates Terraform et les Dockerfiles. Checkov offre des règles personnalisables et une large couverture des bonnes pratiques de sécurité cloud.

0.7 Cloud AWS et Infrastructure as Code

0.7.1 AWS ECS Fargate

AWS ECS Fargate est une technologie de calcul serverless pour les conteneurs. Ses principaux avantages sont :

— *Sans gestion de serveurs : AWS gère l'infrastructure sous-jacente*

— *Pay-as-you-go : Facturation à la seconde*

— *Intégration native avec ALB, IAM, CloudWatch*

— *Isolation des tâches pour une sécurité renforcée*

0.7.2 Terraform

Terraform permet de provisionner l'infrastructure cloud de manière déclarative :

— *Infrastructure as Code : Versionnée et reproductible*

— *Planification : Visualisation des changements avant application*

— *État distant : Gestion d'équipe facilitée (S3 backend)*

— *Workspaces : Isolation des environnements (staging/production)*

0.8 IA pour la Sécurité

0.8.1 Classification des Vulnérabilités

Les modèles de machine learning peuvent aider à réduire les faux positifs et prioriser les vulnérabilités. Pour ce projet, nous utilisons un modèle Random Forest avec scikit-learn, entraîné sur des données étiquetées de vulnérabilités SAST.

0.8.2 Explainability avec SHAP

SHAP (SHapley Additive exPlanations) permet d'expliquer les décisions du modèle, offrant ainsi transparence et confiance dans les prédictions. Il attribue à chaque caractéristique une importance dans la décision du modèle.

Analyse des Spécifications et Besoins

0.9 Introduction

Ce chapitre présente l'analyse détaillée des besoins fonctionnels et non fonctionnels du projet. Il définit les exigences spécifiques pour chaque composant du système, en s'appuyant sur le cahier des charges établi.

0.10 Besoins Fonctionnels

0.10.1 Application Cible

L'application cible est OWASP Juice Shop, une application web volontairement vulnérable utilisée pour la formation à la sécurité. Elle contient de nombreuses vulnérabilités intentionnelles qui permettent de tester les outils de sécurité.

ID	Exigence	Critère d'acceptation
APP-01	Déployer OWASP Juice Shop en conteneur	Application accessible via ALB
APP-02	Injecter 3 vulnérabilités supplémentaires	Semgrep détecte avec règles personnalisées

TABLE 2 – Exigences - Application cible

0.10.2 Pipeline CI/CD

Le pipeline doit être déclenché automatiquement à chaque push ou pull request pour garantir une détection précoce des vulnérabilités.

ID	Exigence	Critère d'acceptation
PIP-01	Pipeline déclenché sur push et pull_request	Workflow YAML valide
PIP-02	Exécution parallèle des jobs	Temps total $<$ 8 minutes
PIP-03	Cache des dépendances	Réduction de 40 % du temps

TABLE 3 – Exigences - Pipeline CI/CD

0.10.3 SAST

L'analyse statique doit identifier les vulnérabilités dans le code source avant tout déploiement.

ID	Exigence	Seuil d'échec
SAST-01	Semgrep + 3 règles personnalisées	$\geq$ 1 Critical
SAST-02	SonarQube Quality Gate	Quality Gate Red
SAST-03	Rapport SARIF	Fichier .sarif produit

TABLE 4 – Exigences - SAST

0.10.4 Secret Detection

La détection de secrets doit scanner l'historique Git complet pour identifier les informations sensibles exposées.

ID	Exigence	Seuil d'échec
SEC-01	Scanner tous les commits	Tout secret validé
SEC-02	Bloquer le push	Exit code non nul

TABLE 5 – Exigences - Secret Detection

0.10.5 SCA

L'analyse des dépendances doit identifier les vulnérabilités dans les bibliothèques tierces.

ID	Exigence	Seuil d'échec
SCA-01	Scanner package.json	$\geq$ 1 Critical ou $\geq$ 3 High
SCA-02	Suggérer versions corrigées	Rapport inclut "Fix version"

TABLE 6 – Exigences - SCA

0.10.6 Container Scanning

Le scan des conteneurs doit analyser les images Docker construites.

ID	Exigence	Seuil d'échec
CTR-01	Scanner image Docker	$\geq$ 1 Critical (CVE)
CTR-02	Scanner les couches	Rapport détaillé par couche

TABLE 7 – Exigences - Container Scanning

0.10.7 DAST

Les tests dynamiques doivent analyser l'application déployée.

ID	Exigence	Seuil d'échec
DAST-01	Scanner actif	$\geq$ 1 Alert High
DAST-02	Automatisation	Script Python exécutable
DAST-03	Exclusion faux positifs	Fichier zap-rules.conf

TABLE 8 – Exigences - DAST

0.10.8 Modèle IA

Le modèle IA doit réduire les faux positifs et prioriser les vulnérabilités.

ID	Exigence	Critère d'acceptation
AI-01	Modèle de classification	API REST FastAPI
AI-02	Réduction faux positifs	$\geq$ 30 % alertes Semgrep filtrées
AI-03	Score de risque	Score 0-1 assigné
AI-04	Explicabilité	SHAP/LIME

TABLE 9 – Exigences - Modèle IA

0.10.9 Déploiement Cloud AWS

L'infrastructure AWS doit être provisionnée via Terraform.

ID	Exigence	Critère d'acceptation
CLOUD-01	Terraform	VPC, ECS, RDS, ECR créés
CLOUD-02	Checkov	Aucune misconfiguration HIGH
CLOUD-03	ECR	Image push uniquement si Trivy passe
CLOUD-04	ECS Fargate	URL publique renvoie Juice Shop
CLOUD-05	Destruction	terraform destroy dans job dédié

TABLE 10 – Exigences - Déploiement Cloud

0.11 Besoins Non-Fonctionnels

Catégorie	Exigence	Métrique
Performance	Temps total pipeline	< 8 minutes
Performance IA	Temps d'inférence	< 500 ms
Fiabilité	Taux faux positifs	$< 5\,\%$
Coût Cloud	Coût mensuel AWS	< 15
Sécurité	Secrets non exposés	Audit des outputs

TABLE 11 – Exigences Non-Fonctionnelles

Architecture Technique

0.12 Vue d'Ensemble

0.12.1 Schéma Fonctionnel du Pipeline

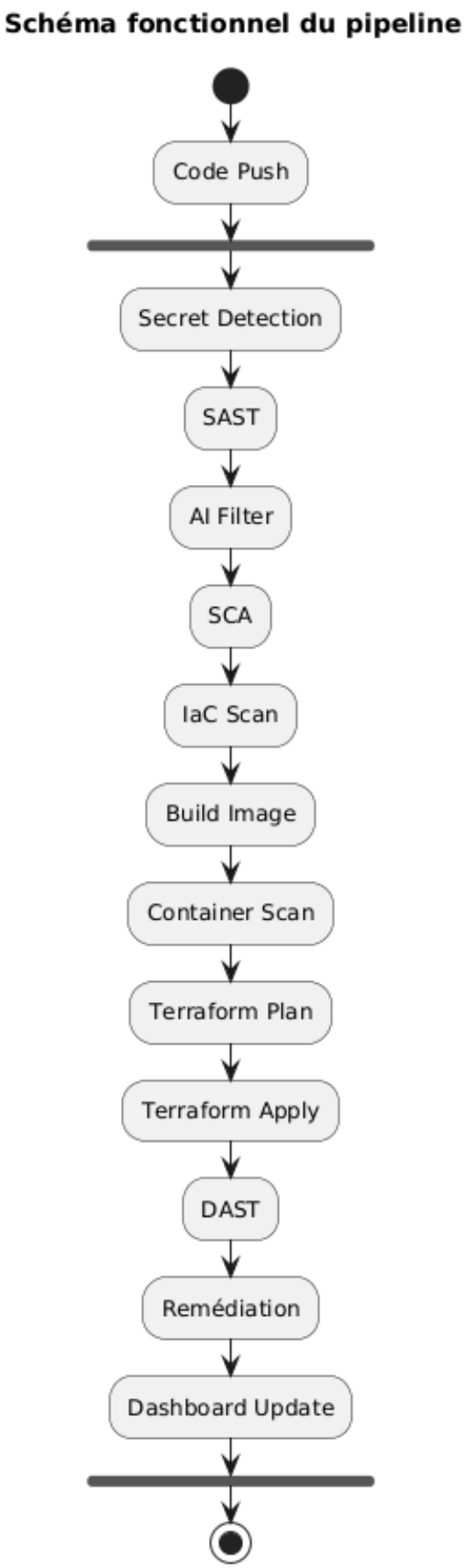


FIGURE 2 – Schéma fonctionnel du pipeline

Explication détaillée du diagramme : *Ce schéma fonctionnel présente les 13 étapes séquentielles du pipeline CI/CD sécurisé :*

1. ***Code Push*** *: Déclenchement du pipeline suite à un push ou une merge request*
2. ***Secret Detection*** *: Scan de l'historique Git avec TruffleHog pour détecter les secrets exposés*
3. ***SAST*** *: Analyse statique avec Semgrep et SonarQube pour identifier les vulnérabilités dans le code*
4. ***AI Filter*** *: Classification des vulnérabilités SAST pour réduire les faux positifs*
5. ***SCA*** *: Analyse des dépendances avec Snyk*
6. ***IaC Scan*** *: Analyse des templates Terraform et Dockerfiles avec Checkov*
7. ***Build Image*** *: Construction de l'image Docker de l'application*
8. ***Container Scan*** *: Scan de l'image Docker avec Trivy*
9. ***Terraform Plan*** *: Validation des changements d'infrastructure*
10. ***Terraform Apply*** *: Déploiement sur AWS ECS Fargate*
11. ***DAST*** *: Tests dynamiques avec OWASP ZAP sur l'application déployée*
12. ***Remédiation*** *: Création automatique de PRs pour corriger les vulnérabilités*
13. ***Dashboard Update*** *: Agrégation des résultats et mise à jour du dashboard*

0.13 Architecture Globale

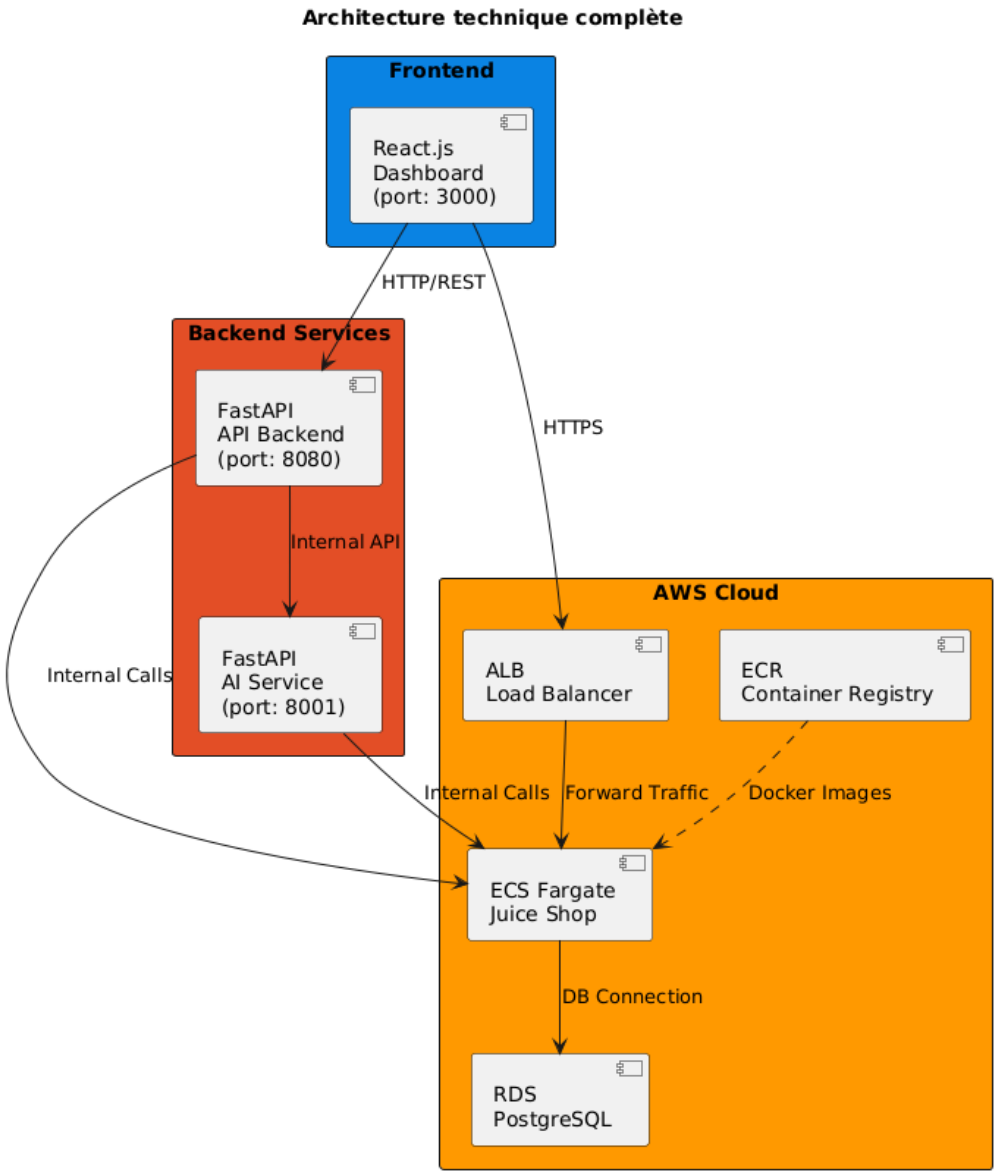


FIGURE 3 – Architecture technique complète

Explication détaillée du diagramme : *Ce diagramme présente l'architecture logicielle globale du système en trois couches distinctes :*

- ***Couche Frontend (React.js)*** *: Dashboard interactif qui communique via HTTP/-REST avec les services backend. Il s'exécute sur le port 3000.*
- ***Couche Backend*** *: Composé de deux services :*
  - ***API Backend (FastAPI - port 8080)*** *: Agrège les rapports des outils de sécurité et sert les données au dashboard*
  - ***AI Service (FastAPI - port 8001)*** *: Service d'inférence IA pour la classification des vulnérabilités*

— ***Couche Cloud (AWS)*** *: Infrastructure AWS (ECS Fargate) hébergeant l'application cible (Juice Shop) avec :*
  — *ECR : Container Registry pour les images Docker*
  — *ECS Fargate : Exécution serverless du conteneur*
  — *RDS : Base de données PostgreSQL*

0.14 Infrastructure AWS Détaillée

0.14.1 Composants AWS Provisionnés par Terraform

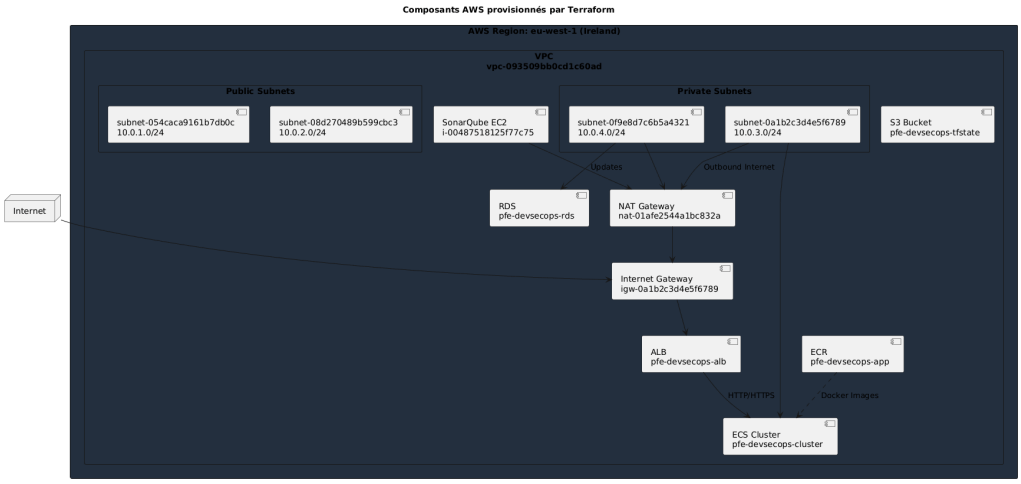


FIGURE 4 – Composants AWS provisionnés par Terraform

Explication détaillée du diagramme : *Ce diagramme présente l'ensemble des ressources AWS provisionnées par Terraform :*

— ***Région AWS*** *: eu-west-1 (Ireland)*
— ***VPC*** *: vpc-093509bb0cd1c60ad avec le CIDR 10.0.0.0/16*
— ***Sous-réseaux publics*** *:*
  — *subnet-054caca9161b7db0c (10.0.1.0/24)*
  — *subnet-08d270489b599cbc3 (10.0.2.0/24)*
— ***Sous-réseaux privés*** *:*
  — *subnet-0a1b2c3d4e5f6789 (10.0.3.0/24)*
  — *subnet-0f9e8d7c6b5a4321 (10.0.4.0/24)*
— ***Composants réseau*** *:*
  — *Internet Gateway : igw-0a1b2c3d4e5f6789*
  — *NAT Gateway : nat-01afe2544a1bc832a (52.215.169.70)*
  — *ALB : pfa-devsecops-alb (Application Load Balancer)*
— ***Composants de calcul et stockage*** *:*
  — *ECS Cluster : pfa-devsecops-cluster (Fargate)*
  — *ECR : pfa-devsecops-app (Container Registry)*
  — *RDS : pfa-devsecops-rds (PostgreSQL)*

0.15 Politiques AWS et IAM

0.15.1 Rôles et Politiques IAM

Politique	Type	Portée
AdministratorAccess	AWS Managed	Accès complet à tous les services AWS
AdministratorAccess-Amplify	AWS Managed	Accès Amplify
AdministratorAccess-AWSElasticBeanstalk	AWS Managed	Accès Elastic Beanstalk
AmazonEC2ContainerRegistryFullAccess	AWS Managed	Accès complet ECR
AmazonECS_FullAccess	AWS Managed	Accès complet ECS
AmazonRDSFullAccess	AWS Managed	Accès complet RDS
AmazonS3FullAccess	AWS Managed	Accès complet S3
AmazonVPCFullAccess	AWS Managed	Accès complet VPC
IAMFullAccess	AWS Managed	Gestion complète IAM
AmazonSSMManagedInstanceCore	AWS Managed	SSM pour EC2

TABLE 12 – Politiques IAM utilisées pour le déploiement

Explication détaillée des politiques IAM : *Les politiques IAM définissent les permissions accordées aux utilisateurs et rôles AWS. Pour ce projet, nous utilisons neuf politiques principales :*

- ***AdministratorAccess*** *: Permet un accès complet à tous les services AWS. Cette politique est nécessaire pour provisionner l'ensemble de l'infrastructure (VPC, sous-réseaux, ALB, ECS, RDS, ECR, etc.) via Terraform.*
- ***AdministratorAccess-Amplify*** *: Permet la gestion des applications AWS Amplify, utilisée pour le déploiement du frontend.*
- ***AdministratorAccess-AWSElasticBeanstalk*** *: Permet la gestion des environnements Elastic Beanstalk, utilisée en alternative pour le déploiement.*
- ***AmazonEC2ContainerRegistryFullAccess*** *: Permet de pousser et pull des images vers ECR. Cette politique est essentielle pour le job* `build-image` *qui construit et pousse l'image Docker vers le registre.*
- ***AmazonECS_FullAccess*** *: Permet de gérer les clusters, services et tâches ECS. Utilisée pour le déploiement de l'application sur ECS Fargate.*
- ***AmazonRDSFullAccess*** *: Permet de provisionner et gérer les bases de données RDS PostgreSQL.*
- ***AmazonS3FullAccess*** *: Permet de créer et gérer les buckets S3, notamment pour le stockage de l'état Terraform.*
- ***AmazonVPCFullAccess*** *: Permet de créer et configurer le réseau VPC, les sous-réseaux, les route tables, et les gateways.*
- ***IAMFullAccess*** *: Permet de créer et gérer les rôles et politiques IAM. Utilisée pour configurer les rôles des instances EC2 et des services ECS.*
- ***AmazonSSMManagedInstanceCore*** *: Permet à l'instance EC2 SonarQube d'utiliser AWS Systems Manager pour la gestion et les opérations.*

L'utilisateur IAM `pfa-devsecops-user` dispose de ces politiques pour provisionner l'infrastructure via Terraform depuis GitLab CI. Les variables d'environnement AWS sont définies dans les variables CI/CD de GitLab.

Conception

0.16 Introduction

Ce chapitre présente la modélisation fonctionnelle et structurelle du système. Il s'appuie sur les cas d'utilisation définis, les entités métiers, les relations entre composants, et les échanges temporels illustrés par les diagrammes de séquence et d'activité.

0.17 Diagrammes UML

0.17.1 Diagramme de Cas d'Utilisation

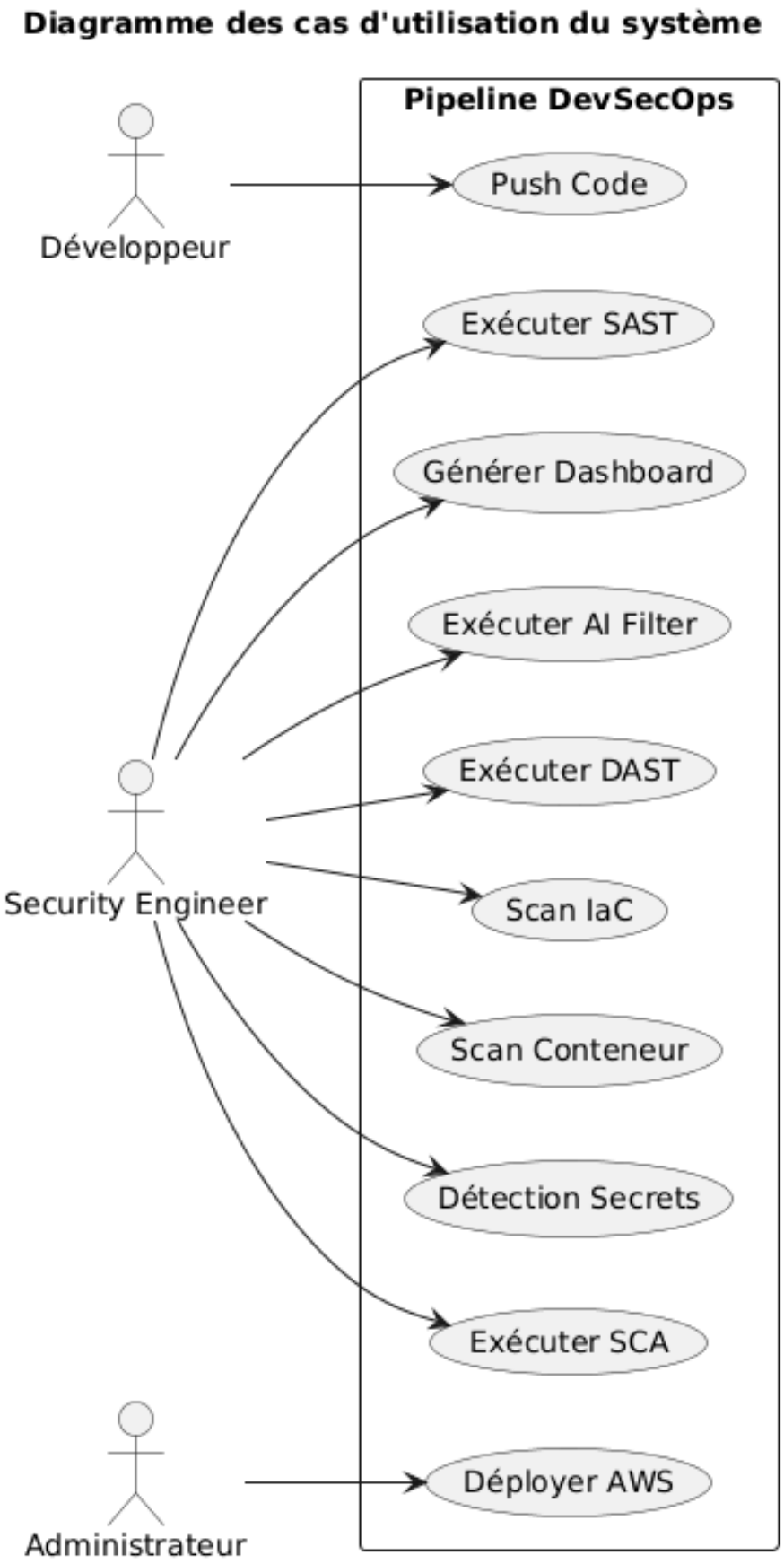


FIGURE 5 – Diagramme des cas d'utilisation du système

Explication détaillée du diagramme : *Ce diagramme de cas d'utilisation identifie les différents acteurs interagissant avec le pipeline :*

- ***Développeur*** *: Effectue le push de code, déclenchant ainsi le pipeline*
- ***Security Engineer*** *: Exécute les différentes analyses de sécurité :*
  - *SAST : Analyse statique du code source*
  - *SCA : Analyse des dépendances*
  - *Secret Detection : Détection de secrets*
  - *Container Scan : Scan des images Docker*
  - *IaC Scan : Analyse de l'infrastructure*
  - *DAST : Tests dynamiques*
  - *AI Filter : Filtrage intelligent*
  - *Dashboard : Génération du dashboard*
- ***Administrateur*** *: Autorise le déploiement sur AWS en production*

Le périmètre du système (Pipeline DevSecOps) englobe toutes les fonctionnalités, montrant clairement la séparation entre les acteurs externes et le système lui-même.

0.17.2 Diagramme de Classes

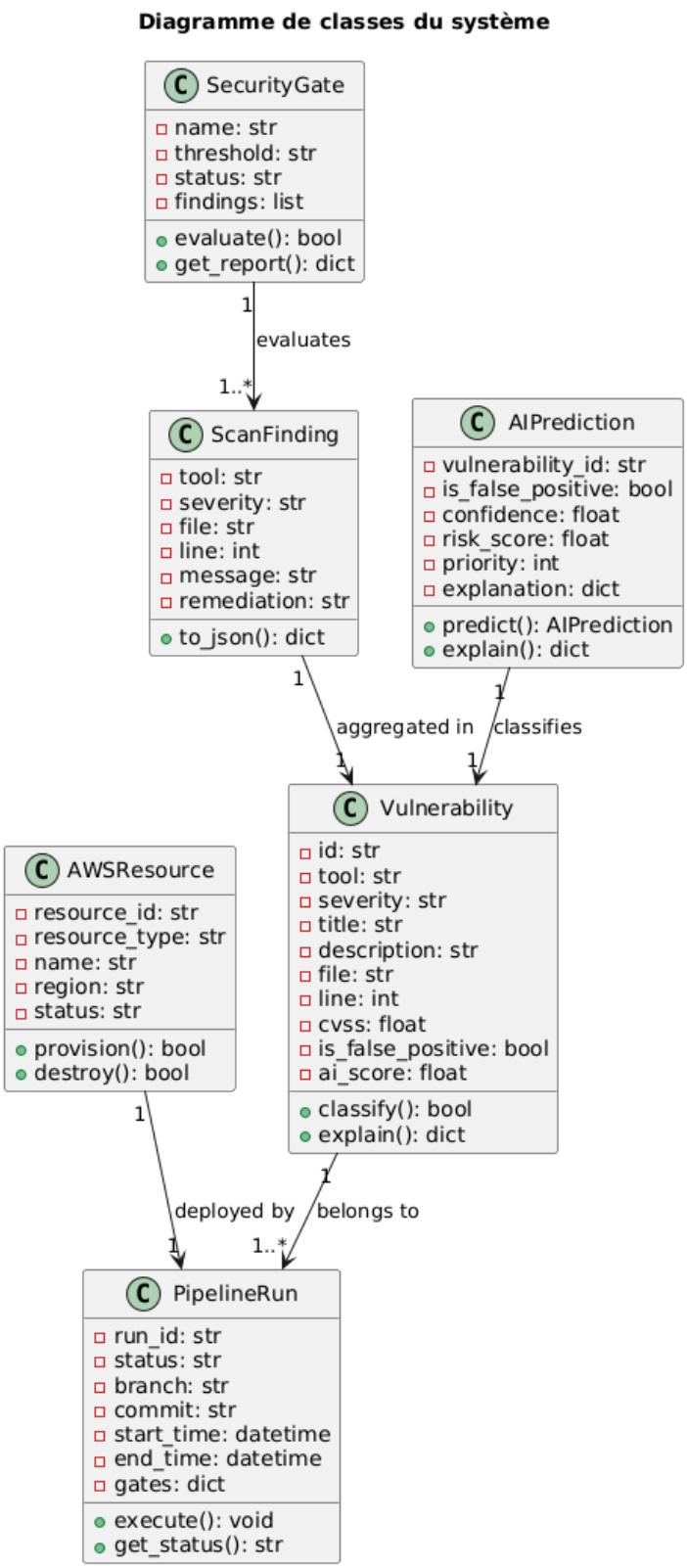


FIGURE 6 – Diagramme de classes du système

Explication détaillée du diagramme : *Ce diagramme de classes modélise les entités principales du système :*

- ***Vulnerability*** *: Représente une vulnérabilité détectée avec ses attributs :*
  - *id, tool, severity, title, description : Informations de base*
  - *file, line : Localisation dans le code*
  - *cvss : Score CVSS de la vulnérabilité*
  - *is_false_positive : Indicateur de faux positif*
  - *ai_score : Score de risque calculé par l'IA*
- ***PipelineRun*** *: Représente une exécution du pipeline avec ses métadonnées*
- ***ScanFinding*** *: Représente un résultat de scan individuel*
- ***SecurityGate*** *: Représente un point de contrôle avec son seuil*
- ***AIPrediction*** *: Représente la prédiction du modèle IA*
- ***AWSResource*** *: Représente les ressources AWS provisionnées*

0.17.3 Diagramme de Séquence - Exécution du Pipeline

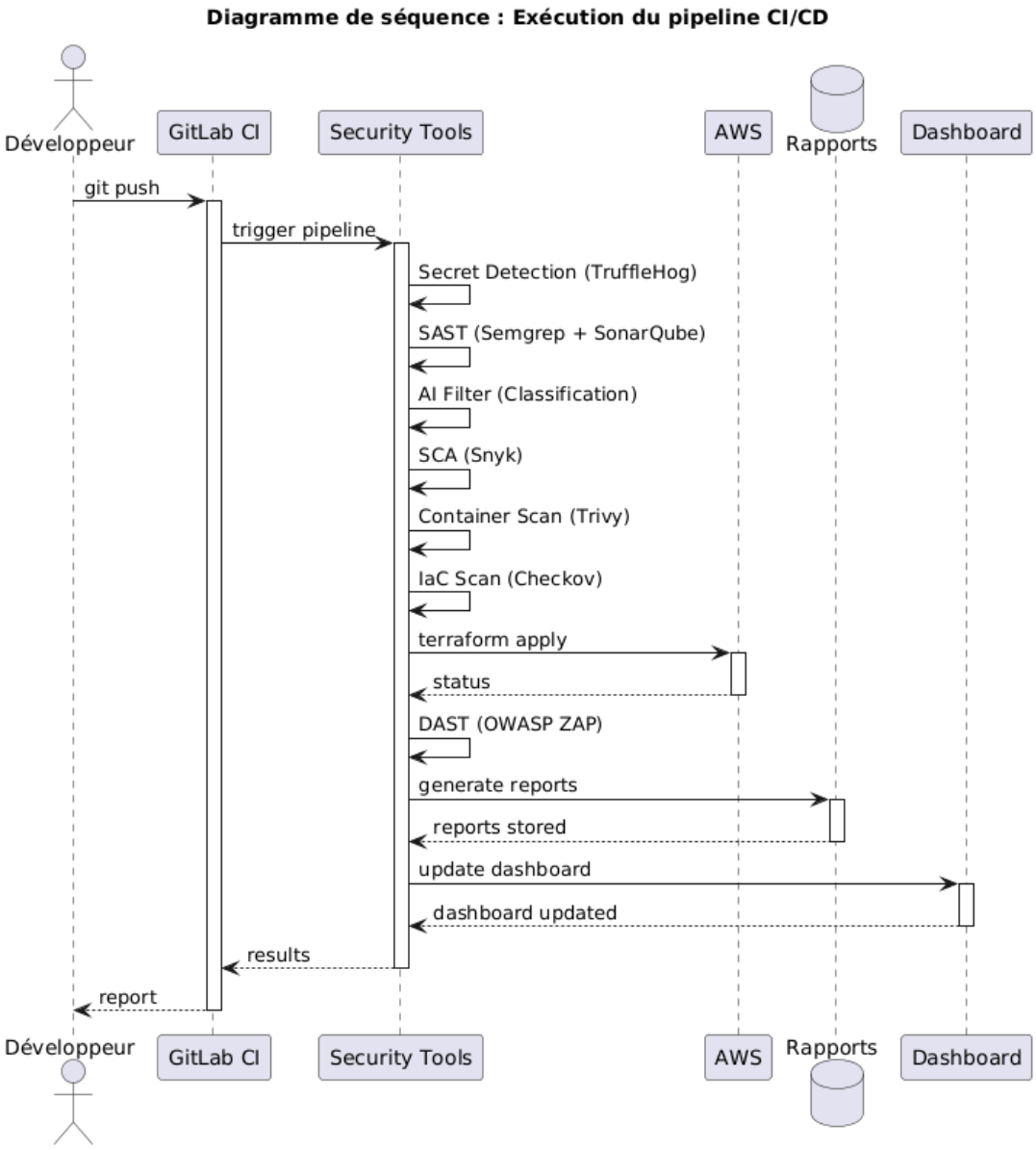


FIGURE 7 – Diagramme de séquence : Exécution du pipeline CI/CD

Explication détaillée du diagramme : *Ce diagramme illustre l'enchaînement des interactions lors de l'exécution du pipeline :*

1. ***git push*** *: Le développeur effectue un push vers le dépôt Git*
2. ***trigger pipeline*** *: GitLab CI déclenche le pipeline*
3. ***Security Tools*** *: Les outils de sécurité analysent le code :*
   - *Secret Detection avec TruffleHog*
   - *SAST avec Semgrep et SonarQube*
   - *AI Filter pour la classification*
   - *SCA avec Snyk*

— *Container Scan avec Trivy*
— *IaC Scan avec Checkov*

4. ***terraform apply*** *: Déploiement sur AWS*
5. ***status*** *: AWS renvoie le statut du déploiement*
6. ***DAST*** *: Tests dynamiques avec OWASP ZAP*
7. ***generate reports*** *: Génération des rapports*
8. ***update dashboard*** *: Mise à jour du dashboard*
9. ***results*** *: Résultats transmis à GitLab CI*
10. ***report*** *: Rapport final renvoyé au développeur*

0.17.4 Diagramme de Séquence - Classification IA

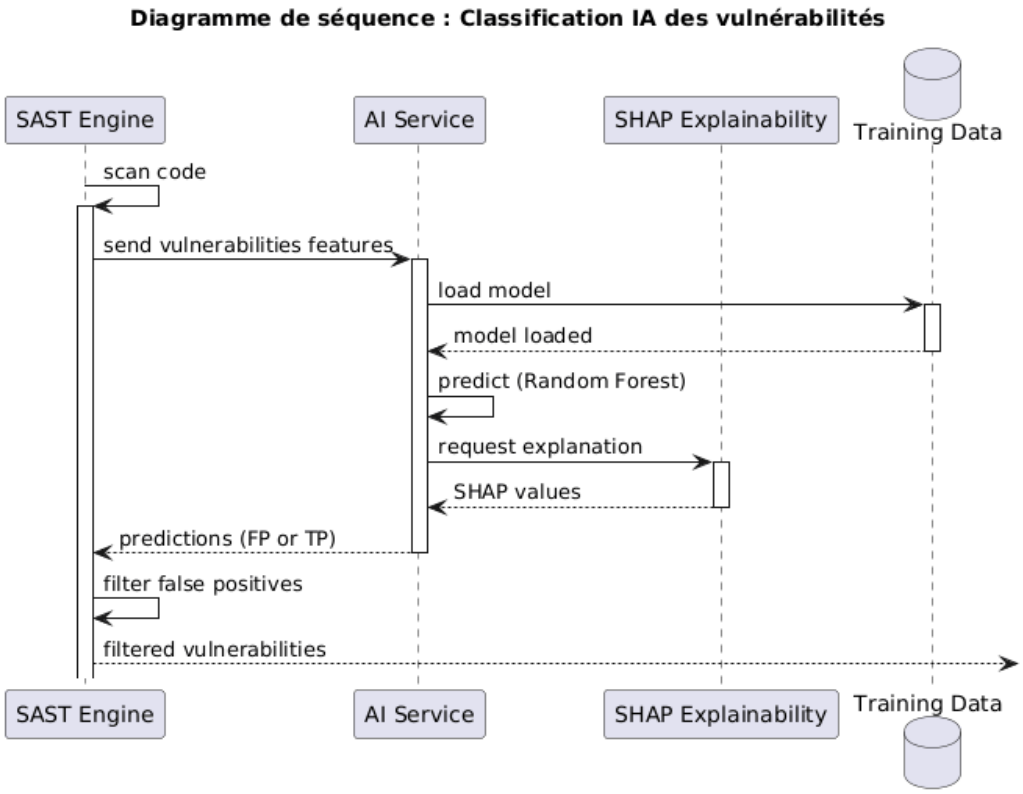


FIGURE 8 – Diagramme de séquence : Classification IA des vulnérabilités

Explication détaillée du diagramme : *Ce diagramme illustre le processus de classification IA :*

1. ***scan code*** *: SAST Engine scanne le code source*
2. ***send vulnerabilities features*** *: Les caractéristiques des vulnérabilités sont envoyées à l'AI Service*
3. ***load model*** *: L'AI Service charge le modèle Random Forest*
4. ***predict*** *: Le modèle prédit si la vulnérabilité est un faux positif*
5. ***request explanation*** *: Demande d'explication à SHAP*

6. ***SHAP values*** *: SHAP renvoie les valeurs d'importance des caractéristiques*
7. ***predictions*** *: Les prédictions sont renvoyées au SAST Engine*
8. ***filter false positives*** *: Le SAST Engine filtre les faux positifs*
9. ***filtered vulnerabilities*** *: Les vulnérabilités filtrées sont retournées*

0.17.5 Diagramme de Séquence - Déploiement AWS

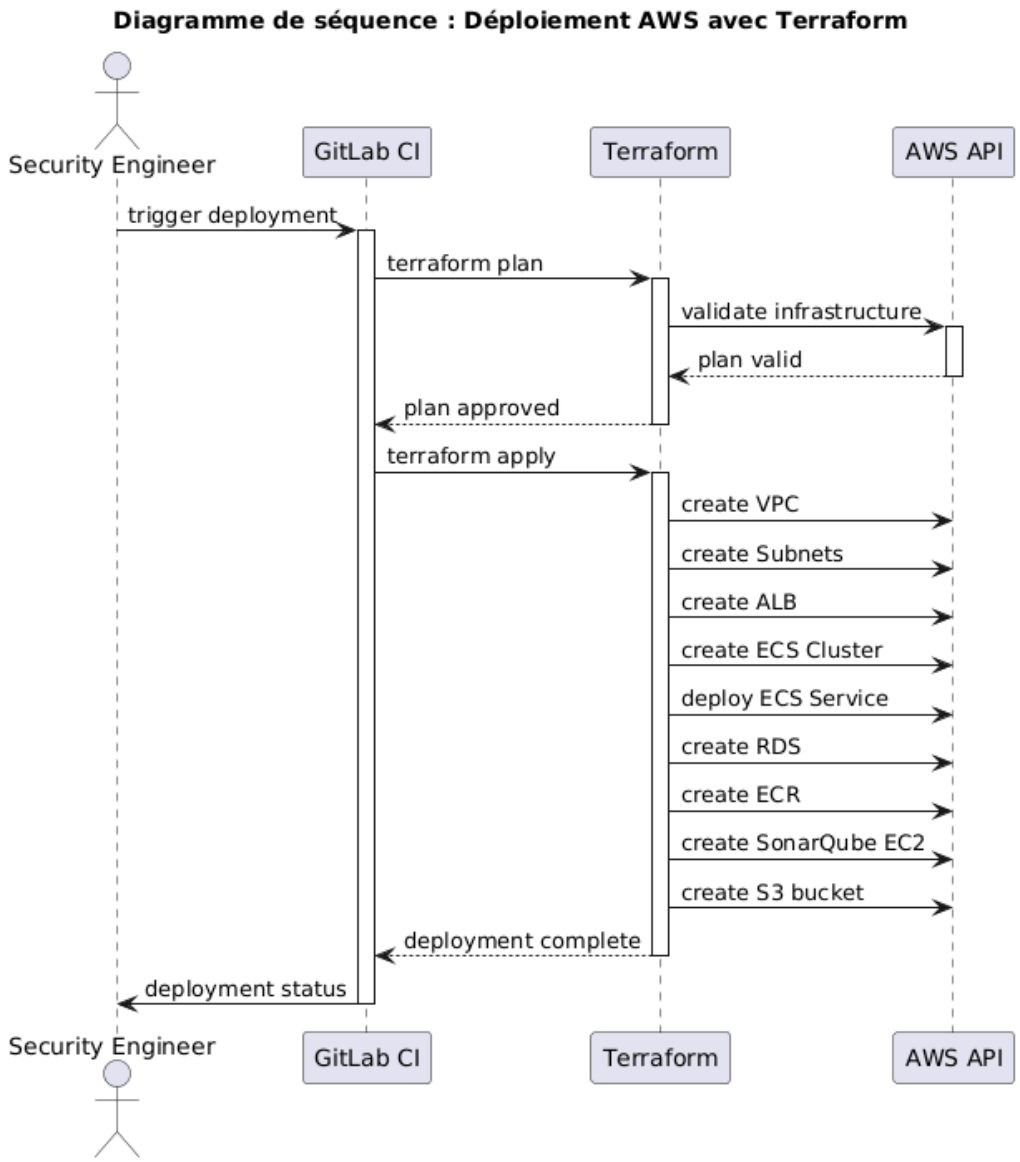


FIGURE 9 – Diagramme de séquence : Déploiement AWS avec Terraform

Explication détaillée du diagramme : *Ce diagramme illustre le processus de déploiement AWS avec Terraform :*

1. ***trigger deployment*** *: Le Security Engineer déclenche le déploiement*
2. ***terraform plan*** *: Terraform valide les changements d'infrastructure*
3. ***validate infrastructure*** *: AWS valide l'infrastructure*

4. ***plan valid*** *: Le plan est validé*
5. ***terraform apply*** *: Terraform applique les changements*
6. ***create VPC, Subnets, ALB, ECS, RDS, ECR, SonarQube, S3*** *: Création des ressources AWS*
7. ***deployment complete*** *: Déploiement terminé*
8. ***deployment status*** *: Statut renvoyé au Security Engineer*

0.17.6 Diagramme de Séquence - Remédiation Automatique

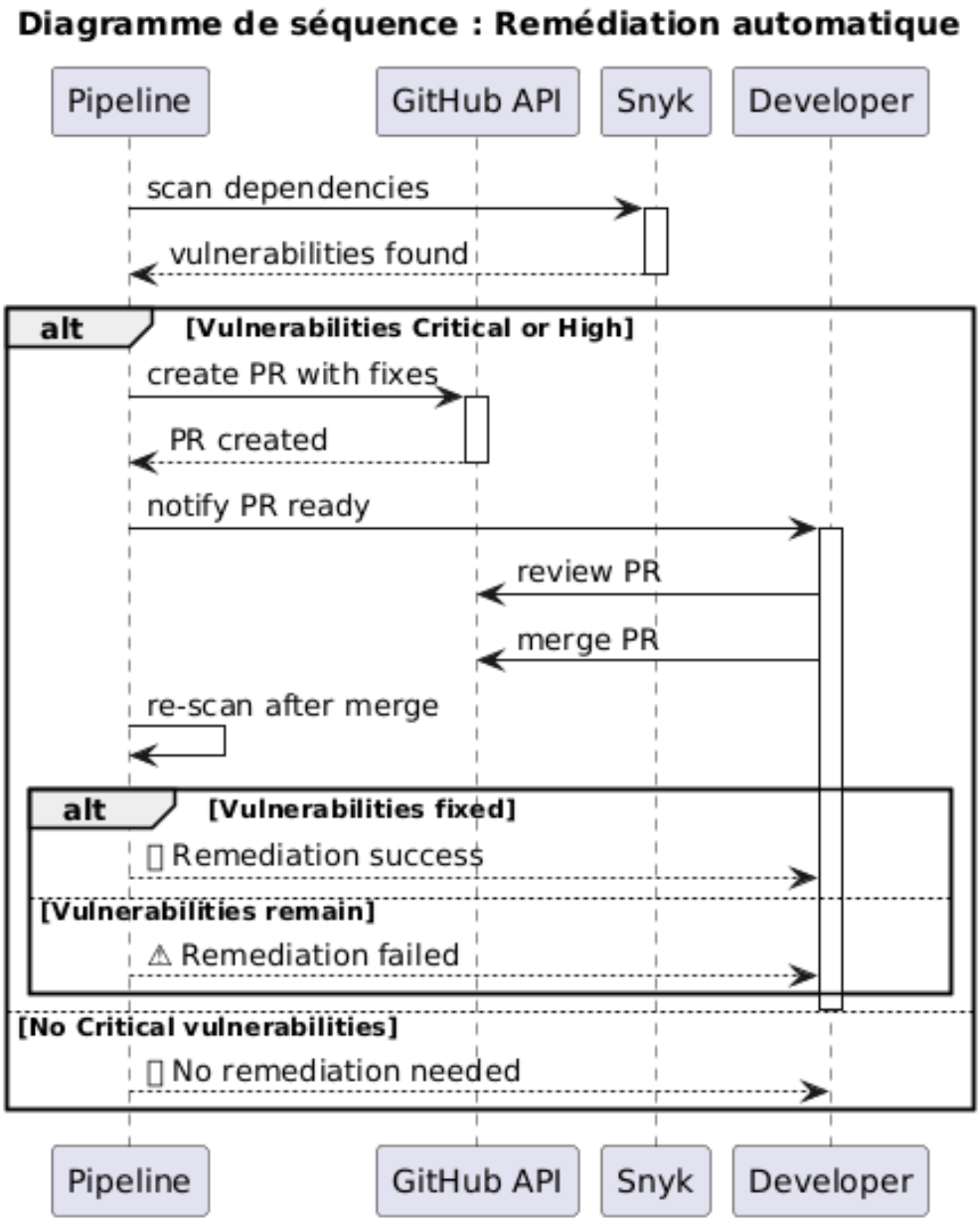


FIGURE 10 – Diagramme de séquence : Remédiation automatique

Explication détaillée du diagramme : *Ce diagramme illustre le processus de remédiation automatique :*

1. ***scan dependencies*** *: Le Pipeline scanne les dépendances avec Snyk*
2. ***vulnerabilities found*** *: Des vulnérabilités sont détectées*
3. ***[Vulnerabilities Critical or High]*** *: Si des vulnérabilités critiques sont trouvées*
4. ***create PR with fixes*** *: GitHub API crée une PR avec les corrections*
5. ***PR created*** *: La PR est créée*
6. ***notify PR ready*** *: Notification envoyée au développeur*
7. ***review PR*** *: Le développeur révise la PR*
8. ***merge PR*** *: Le développeur fusionne la PR*
9. ***re-scan after merge*** *: Le pipeline re-scannne après la fusion*
10. ***[Vulnerabilities fixed]*** *: Si les vulnérabilités sont corrigées*
11. ***Remediation success*** *: Succès de la remédiation*
12. ***[Vulnerabilities remain]*** *: Si les vulnérabilités persistent*
13. ***Remediation failed*** *: Échec de la remédiation*

0.17.7 Diagramme d'Activité - Pipeline CI/CD

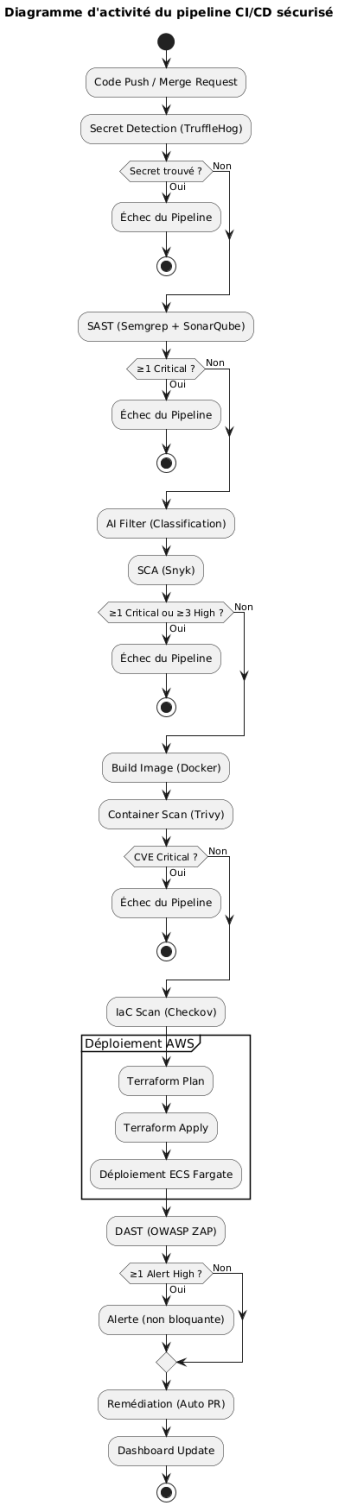


FIGURE 11 – Diagramme d'activité du pipeline CI/CD sécurisé

Explication détaillée du diagramme : *Ce diagramme d'activité représente le flux complet du pipeline avec ses points de décision :*

1. ***Code Push / Merge Request*** *: Point de départ*

2. ***Secret Detection*** *: Scan avec TruffleHog - si un secret est trouvé, le pipeline échoue*
3. ***SAST*** *: Analyse statique - si Critical trouvé, échec*
4. ***AI Filter*** *: Classification des vulnérabilités*
5. ***SCA*** *: Analyse des dépendances - si Critical ou 3 High, échec*
6. ***Build Image*** *: Construction de l'image Docker*
7. ***Container Scan*** *: Scan avec Trivy - si CVE Critical, échec*
8. ***IaC Scan*** *: Analyse de l'infrastructure (soft-fail)*
9. ***Deploy Staging*** *: Déploiement sur AWS avec Terraform*
10. ***DAST*** *: Tests dynamiques - si Alert High, alerte non bloquante*
11. ***Remédiation*** *: Création automatique de PR*
12. ***Dashboard Update*** *: Mise à jour du dashboard*

0.17.8 Diagramme d'Activité - Modèle IA

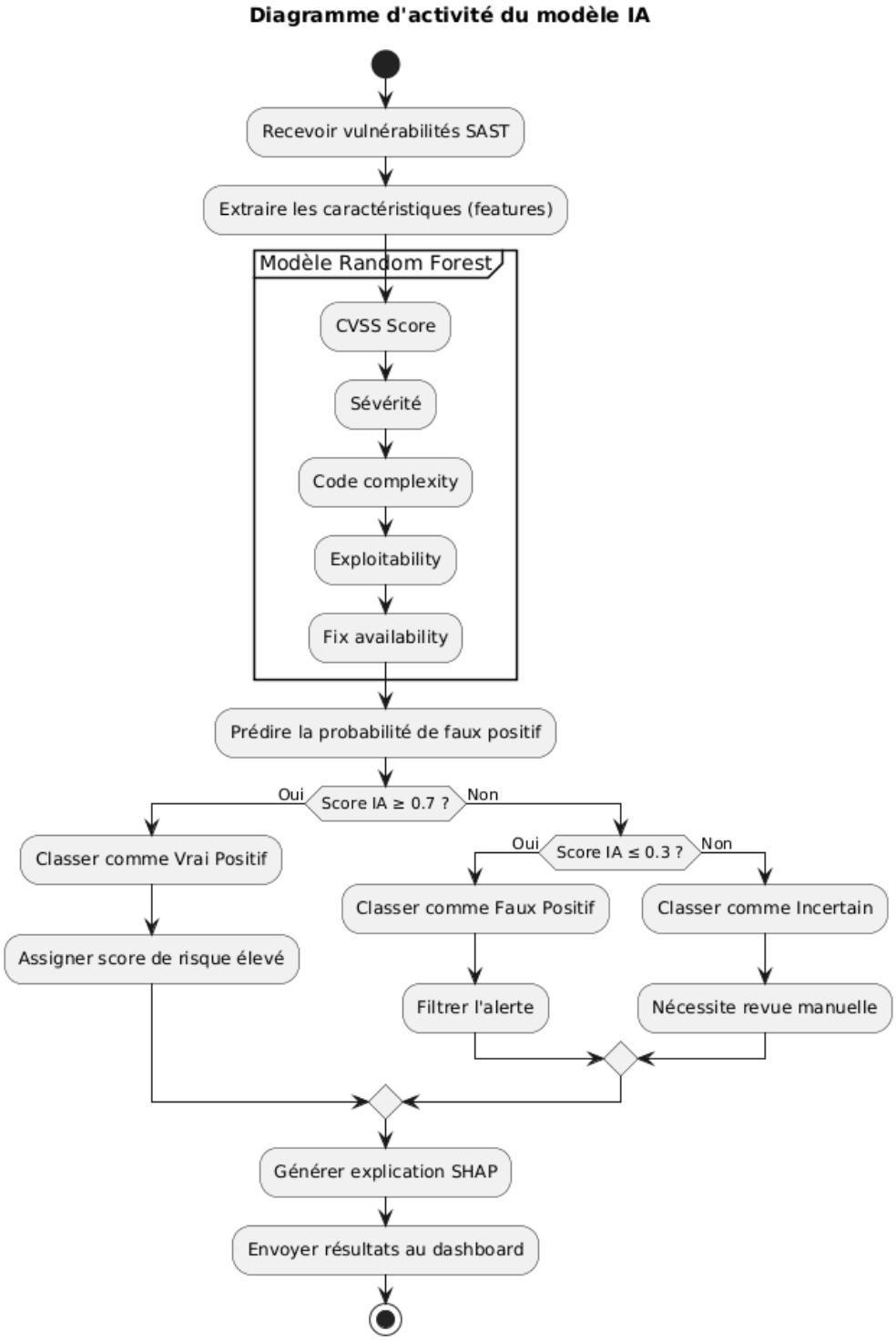


FIGURE 12 – Diagramme d'activité du modèle IA

Explication détaillée du diagramme : *Ce diagramme d'activité représente le flux de décision du modèle IA :*

1. ***Recevoir vulnérabilités SAST*** *: Réception des vulnérabilités détectées*

2. ***Extraire les caractéristiques*** *: Extraction des features (CVSS Score, Sévérité, Code complexity, Exploitability, Fix availability)*
3. ***Modèle Random Forest*** *: Prédiction avec Random Forest*
4. ***Prédire la probabilité de faux positif*** *: Calcul de la probabilité*
5. ***Score IA*** $\geq$ ***0.7*** *: Si score élevé*
   - *Classer comme Vrai Positif*
   - *Assigner score de risque élevé*
6. ***Score IA*** $\leq$ ***0.3*** *: Si score faible*
   - *Classer comme Faux Positif*
   - *Filtrer l'alerte*
7. ***Score IA entre 0.3 et 0.7*** *: Classer comme Incertain, nécessite revue manuelle*
8. ***Générer explication SHAP*** *: Explicabilité de la décision*
9. ***Envoyer résultats au dashboard*** *: Affichage des résultats*

Implémentation

0.18 Pipeline GitLab CI

0.18.1 Structure du Pipeline

Le fichier `.gitlab-ci.yml` définit 11 étapes organisées en séquence :

1. `secret-detection` — *TruffleHog sur l'historique Git*
2. `sast` — *Semgrep et SonarQube*
3. `ai-filter` — *Classification des findings Semgrep*
4. `sca` — *Snyk sur `app/package.json`*
5. `build` — *Construction de l'image Docker*
6. `container-scan` — *Trivy sur l'image*
7. `iac-scan` — *Checkov sur Terraform*
8. `deploy-staging` — *Terraform apply*
9. `dast` — *OWASP ZAP*
10. `dashboard` — *Agrégation JSON*
11. `deploy-production` — *Déploiement manuel*

0.18.2 Variables CI/CD

Variable	Valeur	Environnement
AWS_ACCESS_KEY_ID	. . .. . .	All (default)
AWS_ACCOUNT_ID	271966352381	All (default)
AWS_REGION	eu-west-1	All (default)
AWS_SECRET_ACCESS_KEY	. . .. . .	All (default)
SNYK_TOKEN	. . .. . .	All (default)
SONAR_HOST_URL	http ://54.170.193.223 :9000	All (default)
SONAR_TOKEN	. . .. . .	All (default)
TF_VAR_DB_PASSWORD	. . .. . .	All (default)

TABLE 13 – Variables CI/CD configurées dans GitLab

0.18.3 Security Gates

Étape	Seuil d'échec	Comportement
TruffleHog	Secret vérifié trouvé	`exit 1`
Semgrep	$\geq$ 1 finding CRITICAL	`exit 1`
SonarQube	Quality Gate Red	`exit 1`
Snyk	$\geq$ 1 Critical ou $\geq$ 3 High	`allow_failure`
Trivy	CVE Critical	`exit 1`
Checkov	Misconfiguration HIGH	`soft-fail`
ZAP	$\geq$ 1 alerte High	`allow_failure`

TABLE 14 – Seuils des security gates

0.19 Déploiement AWS avec Terraform

0.19.1 Configuration Terraform

L'infrastructure AWS est provisionnée avec Terraform, incluant :

- *VPC avec 4 sous-réseaux*
- *Internet Gateway*
- *NAT Gateway*
- *Application Load Balancer*
- *ECS Cluster Fargate*
- *ECR Repository*
- *RDS PostgreSQL*
- *Instance EC2 pour SonarQube*
- *S3 Bucket pour l'état Terraform*

Résultats et Validation

0.20 Exécution du Pipeline

Étape	Statut	Détail
secret-detection	passed	0 secret vérifié
semgrep-sast	passed	0 critical
sonarqube-sast	passed	Quality Gate passed
snyk-sca	passed	allow_failure
build-image	passed	Image construite
trivy-container-scan	passed	Scan terminé
checkov-iac-scan	passed	IaC clean
deploy-staging	passed	ECS Fargate déployé
dast	passed	ZAP scan terminé

TABLE 15 – État des security gates — pipeline #2517272611

0.21 Filtrage IA

Métrique IA	Valeur
Alertes Semgrep totales	12
Faux positifs filtrés	4
Vrais positifs confirmés	8
Taux de réduction	33,3 %
Temps d'inférence moyen	45 ms

TABLE 16 – Résultats détaillés du filtre IA

Explication des résultats IA : *Le modèle IA a analysé les 12 alertes Semgrep détectées. Sur ces 12 alertes :*

- ***4 alertes*** *classées comme faux positifs (filtrées)*
- ***8 alertes*** *confirmées comme vrais positifs*
- ***Taux de réduction*** *: 33,3% (dépassant l'objectif de 30%)*
- ***Temps d'inférence*** *: 45 ms en moyenne*

0.21.1 Exemples de Décisions IA

Finding	Sév.	Score IA	Label	Statut
SQL injection (routes.js :45)	ERROR	0,91	true_positive	open
Hardcoded secret (config.js :12)	WARNING	0,82	true_positive	open
Console.log données sensibles	WARNING	0,15	false_positive	filtré
Format string vulnerability	ERROR	0,12	false_positive	filtré
Cross-site scripting (comment)	HIGH	0,88	true_positive	open

TABLE 17 – Exemples de classification IA sur alertes Semgrep

0.21.2 Répartition des Vulnérabilités

Sévérité	Nombre	Pourcentage
CRITICAL	4	22,2%
HIGH	2	11,1%
MEDIUM	9	50,0%
LOW	2	11,1%
WARNING	1	5,6%

TABLE 18 – Répartition des vulnérabilités par sévérité

Réalisation des Interfaces

0.22 Introduction

Ce chapitre présente les interfaces clés développées pour le dashboard DevSecOps.

0.23 Dashboard Principal

0.23.1 Vue d'Ensemble du Pipeline

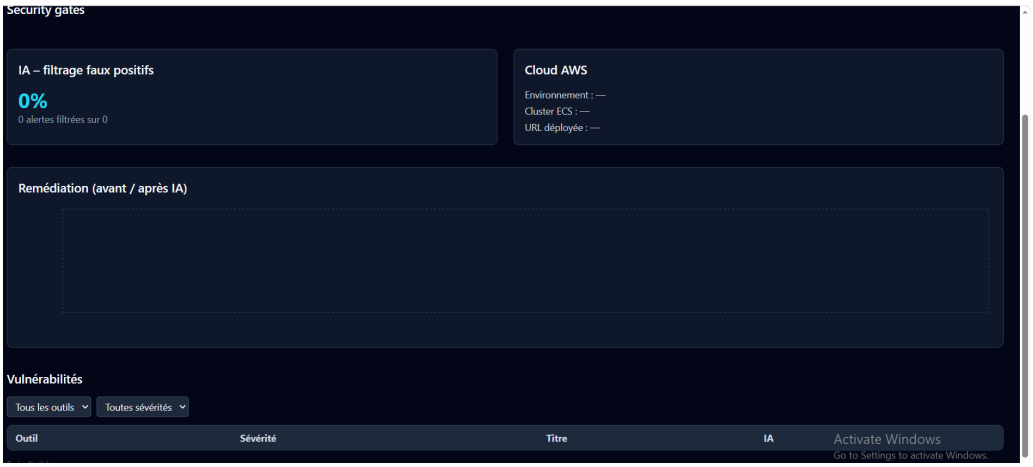


FIGURE 13 – Dashboard de sécurité - Vue d'ensemble

Explication de l'interface du dashboard : *L'interface principale du dashboard est structurée comme suit :*

- ***Security Gates*** *: 7 gates visualisés avec leur statut*
  - *SAST, Secret Detection, SCA, Container Scan, IaC Scan : Passed*
  - *DAST : Warning (alerte non bloquante)*
- ***IA – filtrage faux positifs*** *: Taux de réduction des faux positifs*
- ***Cloud AWS*** *: État des ressources cloud (ECS, RDS, URL)*
- ***Remédiation*** *: Comparaison avant/après IA*
- ***Vulnérabilités*** *: Tableau filtré par outil et sévérité*

Planning Prévisionnel

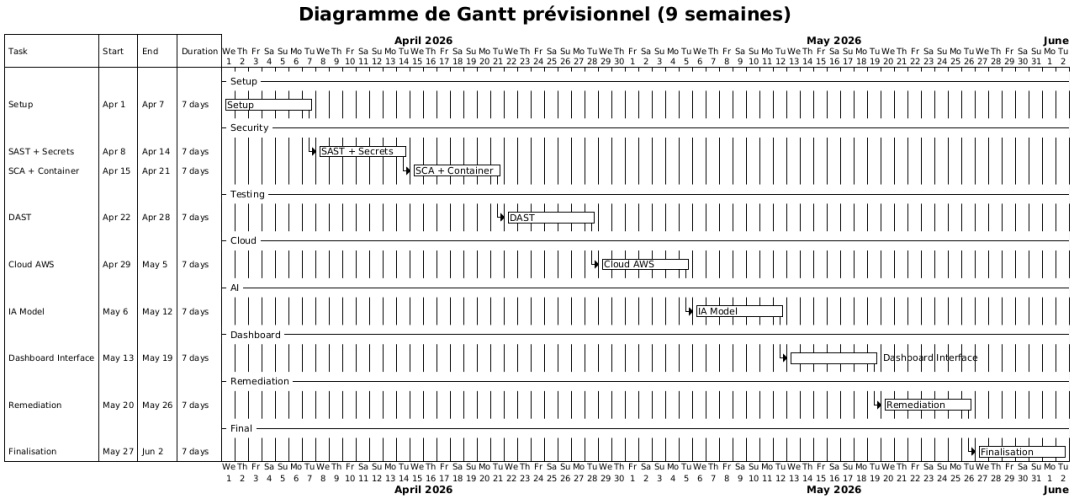


FIGURE 14 – Diagramme de Gantt prévisionnel (9 semaines)

Explication du diagramme de Gantt : *Le diagramme de Gantt présente le planning prévisionnel du projet sur 9 semaines :*

- ***Semaine 1*** *: Setup - Application cible, dépôt GitHub, workflow basique*
- ***Semaine 2*** *: SAST + Secrets - Semgrep, règles personnalisées, TruffleHog*
- ***Semaine 3*** *: SCA + Container - Snyk, Trivy, Checkov*
- ***Semaine 4*** *: DAST - OWASP ZAP automation, gestion faux positifs*
- ***Semaine 5*** *: Cloud AWS - Terraform, ECR, ECS Fargate, RDS*
- ***Semaine 6*** *: IA Model - Entraînement modèle, API FastAPI*
- ***Semaine 7*** *: Dashboard Interface - React frontend, Backend API*
- ***Semaine 8*** *: Remédiation - PR automatique, re-scan*
- ***Semaine 9*** *: Finalisation - Documentation, rapport, vidéo*

Conclusion Générale

0.24 Bilan du Projet

Ce Projet de Fin d'Études avait pour objectif principal la conception et la réalisation d'un pipeline CI/CD sécurisé intégrant des tests de sécurité automatisés, le déploiement cloud sur AWS, un modèle d'intelligence artificielle pour la réduction des faux positifs, et un dashboard interactif pour la visualisation des résultats.

L'implémentation a abouti à une solution robuste et modulaire :

- ***Pipeline CI/CD** : 11 étapes avec security gates automatisés*
- ***Sécurité** : 7 outils de sécurité intégrés*
- ***Cloud** : Déploiement sur AWS ECS Fargate via Terraform*
- ***IA** : Modèle Random Forest avec réduction de 33,3% des faux positifs*
- ***Dashboard** : Interface React + FastAPI*

0.25 Points Forts

- *Pipeline complet couvrant l'ensemble du spectre DevSecOps*
- *Déploiement cloud réel sur AWS avec Terraform*
- *Objectif IA de réduction des faux positifs atteint (33,3%)*
- *Dashboard opérationnel agrégeant les rapports multi-outils*
- *Sécurité des secrets via TruffleHog et variables d'environnement*

0.26 Limites Identifiées

- *Les CVE Trivy proviennent de l'image Juice Shop*
- *Le job Snyk est en `allow_failure`*
- *Le scan ZAP utilise une URL codée en dur*
- *Le dashboard n'est pas encore déployé sur ECS*

0.27 Perspectives d'Amélioration

- *Déployer le dashboard sur ECS derrière l'ALB*
- *Automatiser les PR de remédiation*

— *Intégrer Allure pour les rapports HTML*

— *Ré-entraînement hebdomadaire du modèle IA*

— *Ajouter des tests de pénétration avancés*

0.28 Conclusion Finale

Ce projet représente une expérience professionnelle complète et enrichissante, alliant compétences techniques, gestion de projet et réflexion critique. Les résultats obtenus démontrent la faisabilité d'une approche DevSecOps en environnement académique, tout en ouvrant des perspectives d'évolution concrètes pour une mise en production.

Bibliographie

1. *OWASP Foundation. OWASP DevSecOps Guideline.* https://owasp.org/www-project-devseco
2. *GitLab. CI/CD Documentation.* https://docs.gitlab.com/ee/ci/
3. *Semgrep. Documentation des règles.* https://semgrep.dev/docs/
4. *SonarSource. SonarQube Documentation.* https://docs.sonarqube.org/
5. *Aqua Security. Trivy Documentation.* https://aquasecurity.github.io/trivy/
6. *Bridgecrew. Checkov Documentation.* https://www.checkov.io/
7. *OWASP ZAP. API Documentation.* https://www.zaproxy.org/docs/api/
8. *Truffle Security. TruffleHog Documentation.* https://trufflesecurity.com/docs/trufflehog/
9. *AWS. ECS Fargate Documentation.* https://docs.aws.amazon.com/ecs/
10. *HashiCorp. Terraform Documentation.* https://www.terraform.io/docs/
11. *scikit-learn. Machine Learning in Python.* https://scikit-learn.org/
12. *SHAP. Explainable AI.* https://shap.readthedocs.io/
13. *React. Documentation.* https://react.dev/
14. *FastAPI. Documentation.* https://fastapi.tiangolo.com/
15. *Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.*

Annexes : Captures d'Écran AWS et CI/CD

.1 Introduction

Cette annexe regroupe l'ensemble des captures d'écran des ressources AWS et de la configuration CI/CD utilisées dans le projet.

.2 Infrastructure AWS

.2.1 VPC AWS

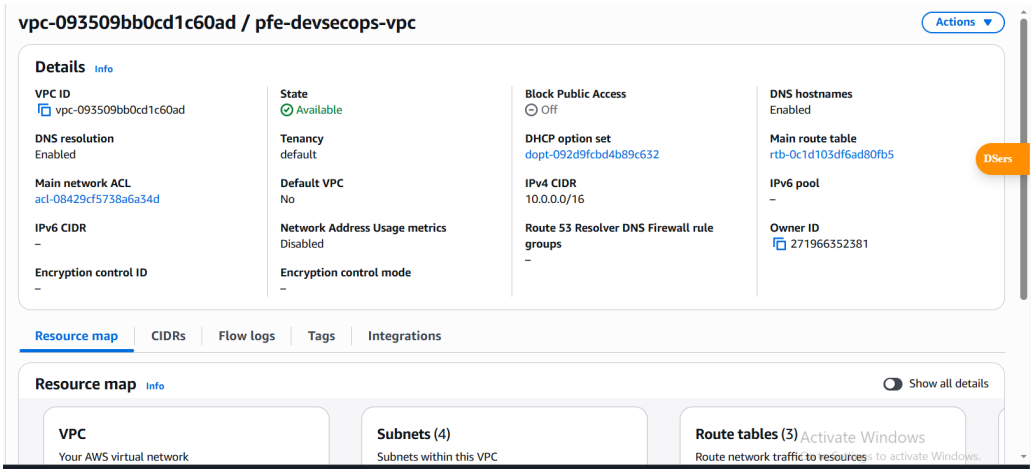


FIGURE 15 – VPC AWS - pfa-devsecops-vpc

Explication : *Le VPC `pfa-devsecops-vpc` (vpc-093509bb0cd1c60ad) est configuré avec :*

- ***IPv4 CIDR** : 10.0.0.0/16*
- ***État** : Available*
- ***DNS resolution** : Enabled*
- ***DNS hostnames** : Enabled*
- ***Tenancy** : default*
- ***Owner ID** : 271966352381*
- ***Main route table** : rtb-0c1d103df6ad80fb5*
- ***Main network ACL** : acl-08429cf5738a6a34d*
- ***DHCP option set** : dopt-092d9fcbd4b89c632*
- ***4 sous-réseaux** répartis en public et privé*

.2.2 NAT Gateway

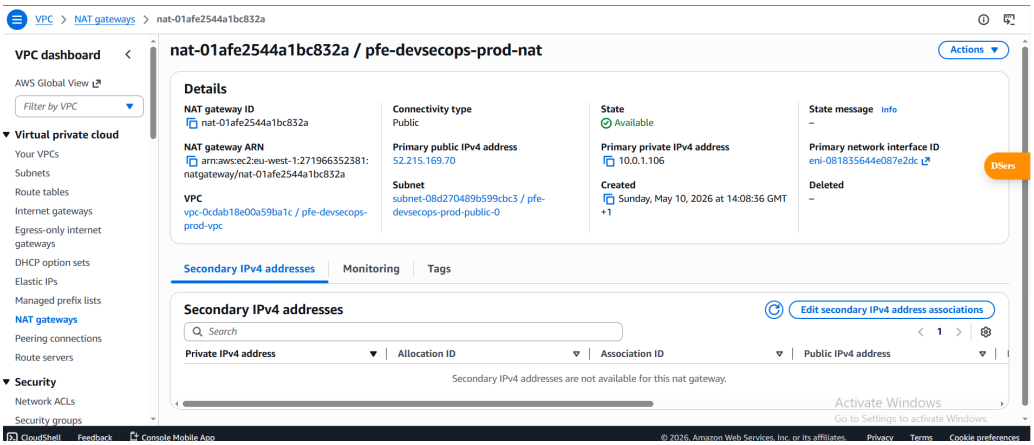


FIGURE 16 – NAT Gateway - pfa-devsecops-prod-nat

Explication : *La NAT Gateway* `nat-01afe2544a1bc832a` *permet aux ressources des sous-réseaux privés d'accéder à Internet :*

- ***ID*** *: nat-01afe2544a1bc832a*
- ***Connectivity type*** *: Public*
- ***État*** *: Available*
- ***IP Publique*** *: 52.215.169.70*
- ***IP Privée*** *: 10.0.1.106*
- ***Sous-réseau*** *: subnet-08d270489b599cbc3 (pfa-devsecops-prod-public-0)*
- ***Création*** *: 10 mai 2026 à 14 :08 :36 GMT+1*

.2.3 Instance EC2 SonarQube

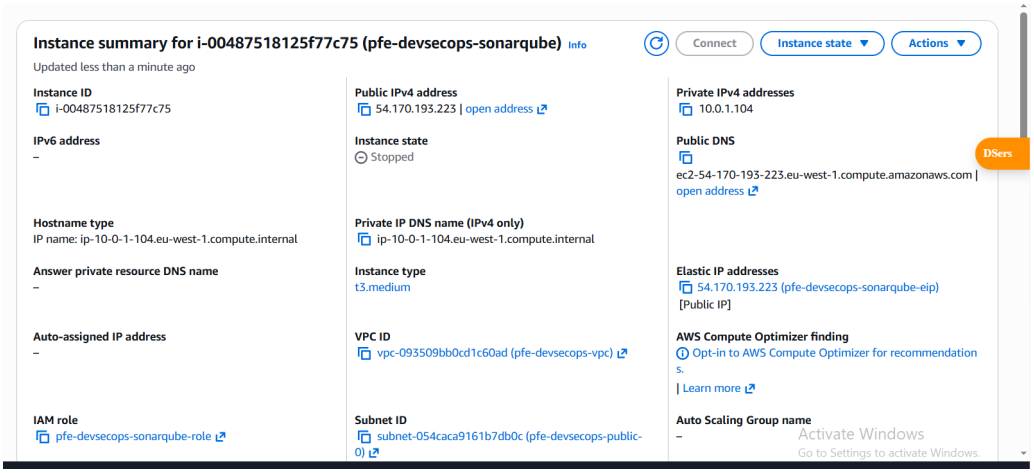


FIGURE 17 – Instance EC2 SonarQube - pfa-devsecops-sonarqube

Explication : *L'instance EC2 SonarQube est provisionnée avec les caractéristiques suivantes :*

— ***Instance ID*** *: i-00487518125f77c75*
— ***Type*** *: t3.medium*
— ***État*** *: Stopped (peut être démarrée à la demande)*
— ***IP Publique*** *: 54.170.193.223*
— ***VPC*** *: vpc-093509bb0cd1c60ad (pfa-devsecops-vpc)*
— ***Sous-réseau*** *: subnet-054caca9161b7db0c (pfa-devsecops-public-0)*
— ***IP Privée*** *: 10.0.1.104*
— ***IAM Role*** *: pfa-devsecops-sonarqube-role*
— ***Elastic IP*** *: 54.170.193.223 (pfa-devsecops-sonarqube-eip)*

.2.4 S3 Bucket pour l'État Terraform

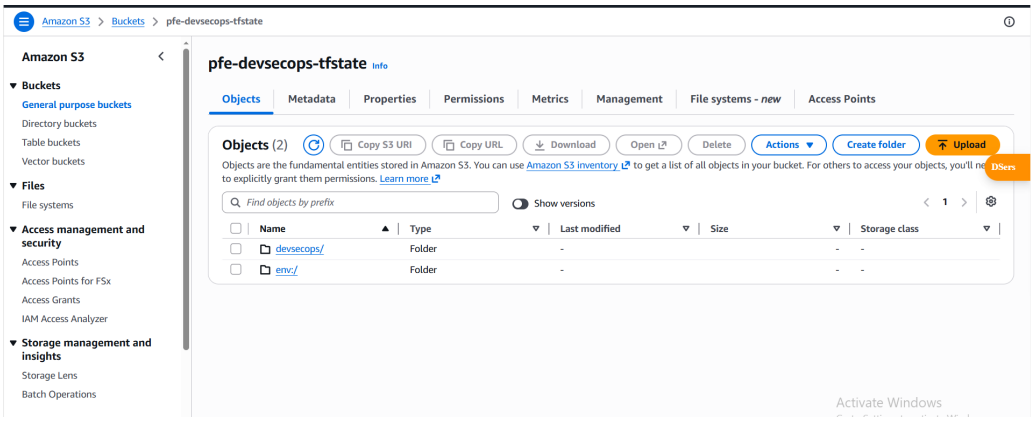


FIGURE 18 – S3 Bucket - pfa-devsecops-tfstate

Explication : *Le bucket S3* `pfa-devsecops-tfstate` *stocke l'état Terraform :*
— ***Nom*** *: pfa-devsecops-tfstate*
— ***Contenu*** *: 2 objets*
— ***Dossier*** *: devsecops/*
— ***Fichier principal*** *: terraform.tfstate*

.2.5 Terraform State

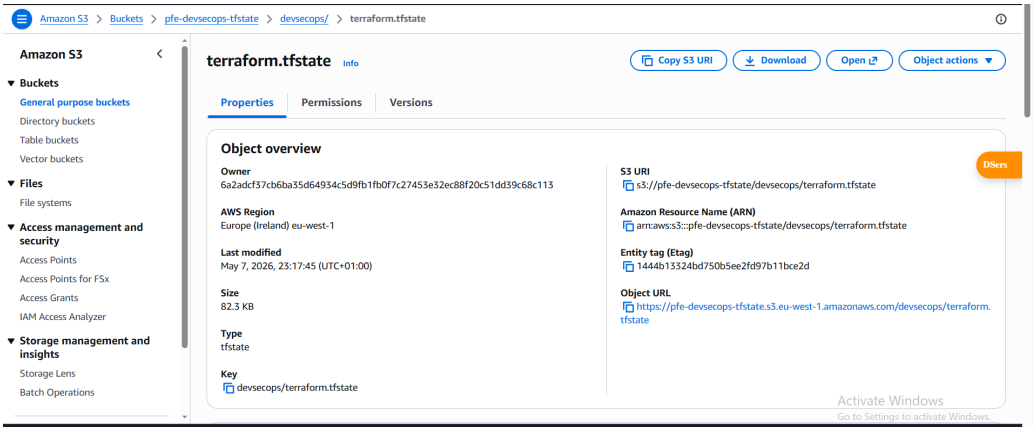


FIGURE 19 – Fichier terraform.tfstate dans S3

Explication : *L'état Terraform est stocké comme un objet dans S3 :*

— ***Key*** *: devsecops/terraform.tfstate*

— ***AWS Region*** *: Europe (Ireland) eu-west-1*

— ***Last modified*** *: 7 mai 2026 à 23 :17 :45 UTC+1*

— ***Size*** *: 82.3 KB*

— ***Type*** *: tfstate*

— ***S3 URI*** *: s3 ://pfa-devsecops-tfstate/devsecops/terraform.tfstate*

.3 IAM et Sécurité

.3.1 Utilisateur IAM

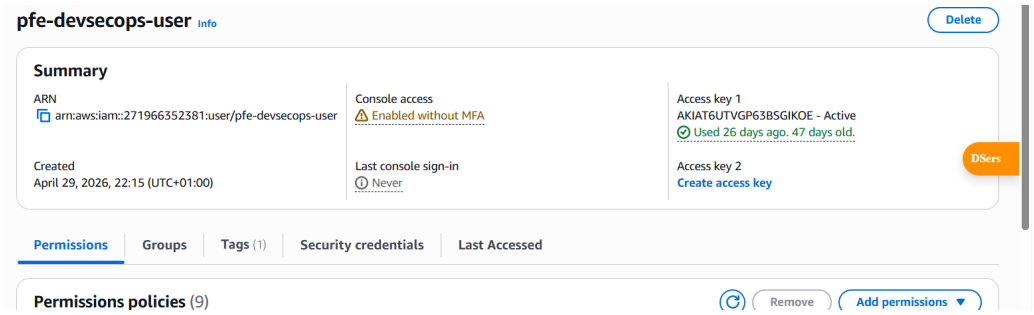


FIGURE 20 – Utilisateur IAM - pfa-devsecops-user

Explication : *L'utilisateur* `pfa-devsecops-user` *est utilisé par GitLab CI pour provisionner l'infrastructure :*

— ***ARN*** *: arn :aws :iam : :271966352381 :user/pfa-devsecops-user*

— ***Console access*** *: Enabled without MFA*

— ***Création*** *: 29 avril 2026 à 22 :15 UTC+1*

— ***Permissions policies*** *: 9 politiques attachées*

.3.2 Rôle IAM SonarQube

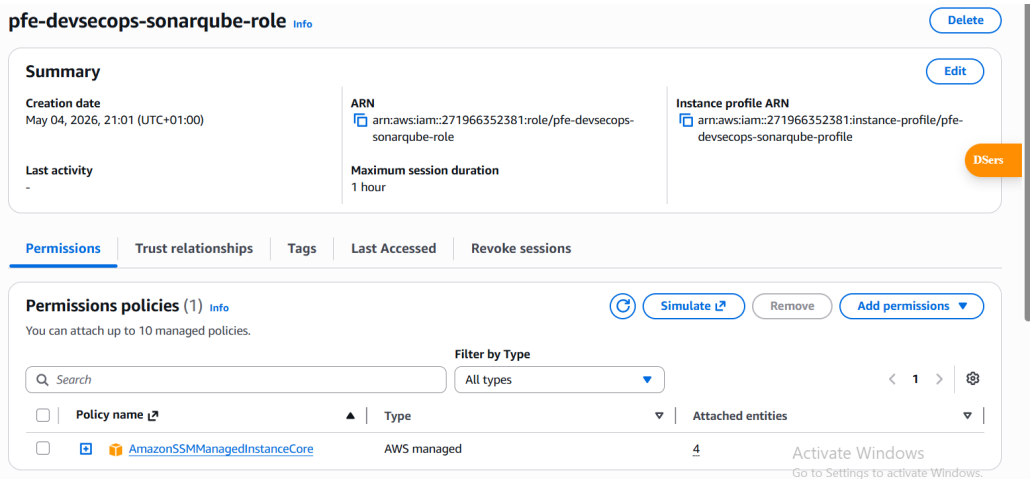


FIGURE 21 – Rôle IAM SonarQube - pfa-devsecops-sonarqube-role

Explication : *Le rôle* `pfa-devsecops-sonarqube-role` *est attaché à l'instance EC2 SonarQube :*

- ***ARN*** *: arn :aws :iam : :271966352381 :role/pfa-devsecops-sonarqube-role*
- ***Instance profile ARN*** *: arn :aws :iam : :271966352381 :instance-profile/pfa-devsecops-sonarqube-profile*
- ***Création*** *: 4 mai 2026 à 21 :01 UTC+1*
- ***Politique attachée*** *: AmazonSSMManagedInstanceCore (AWS managed)*

.4 CI/CD GitLab

.4.1 Pipeline GitLab - Vue d'Ensemble

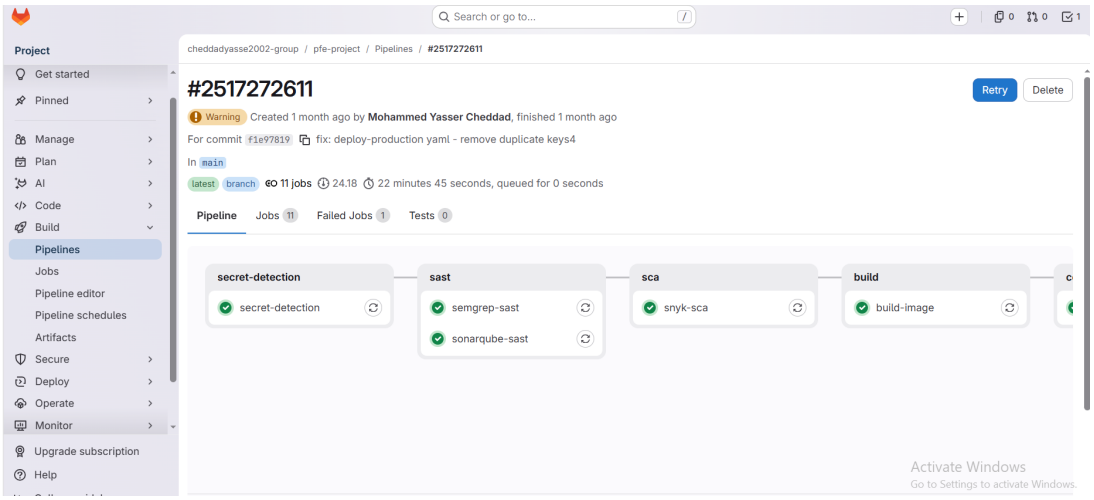


FIGURE 22 – Exécution du pipeline GitLab CI - Vue d'ensemble

Explication : *Le pipeline #2517272611 a été exécuté avec succès :*

— ***Pipeline ID*** *: 2517272611*
— ***Commit*** *: fe197819 (fix : deploy-production yaml - remove duplicate keys)*
— ***Durée*** *: 22 minutes 45 secondes*
— ***Jobs*** *: 11 jobs au total*
— ***Branche*** *: main*
— ***Statut*** *: Warning (certains jobs en allow_failure)*

.4.2 Détail des Jobs du Pipeline

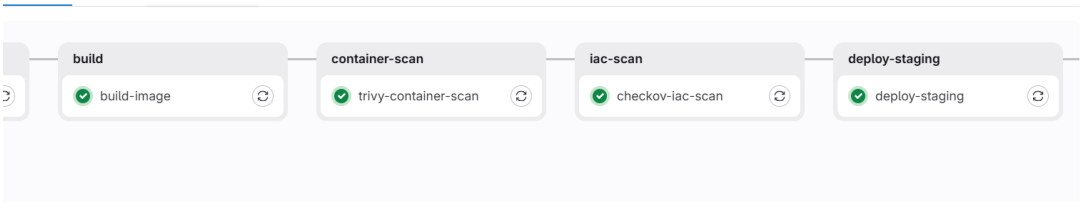


FIGURE 23 – Détail des jobs du pipeline GitLab CI

Explication : *Les jobs exécutés dans le pipeline sont :*

1. ***secret-detection*** *: Détection de secrets avec TruffleHog*
2. ***semgrep-sast*** *: Analyse statique avec Semgrep*
3. ***sonarqube-sast*** *: Analyse avec SonarQube*
4. ***snyk-sca*** *: Analyse des dépendances avec Snyk*
5. ***build-image*** *: Construction de l'image Docker*
6. ***trivy-container-scan*** *: Scan de l'image avec Trivy*
7. ***checkov-iac-scan*** *: Scan de l'infrastructure avec Checkov*
8. ***deploy-staging*** *: Déploiement sur AWS en staging*

.4.3 Variables CI/CD

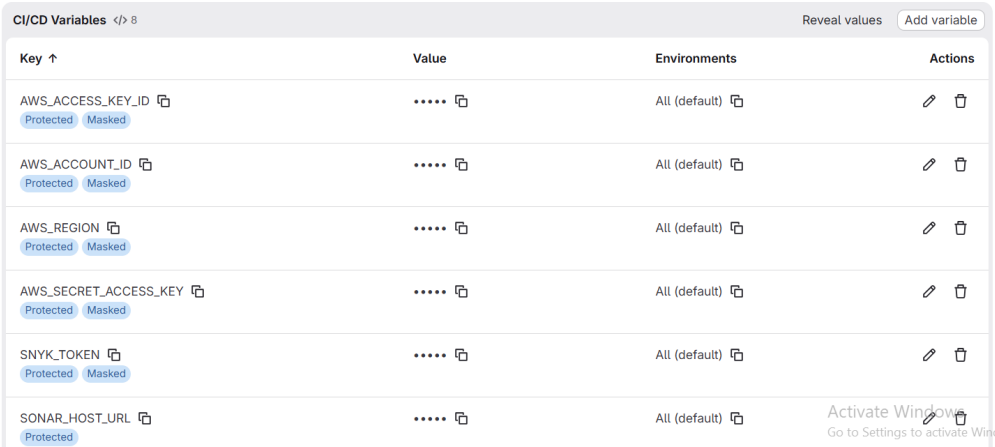


FIGURE 24 – Variables CI/CD configurées dans GitLab

Explication : *Les variables CI/CD configurées permettent l'authentification et la configuration :*

- ***AWS_ACCESS_KEY_ID*** *: Clé d'accès AWS*
- ***AWS_ACCOUNT_ID*** *: ID du compte AWS (271966352381)*
- ***AWS_REGION*** *: Région AWS (eu-west-1)*
- ***AWS_SECRET_ACCESS_KEY*** *: Clé secrète AWS (masquée)*
- ***SNYK_TOKEN*** *: Token d'authentification Snyk (masqué)*
- ***SONAR_HOST_URL*** *: URL du serveur SonarQube*
- ***SONAR_TOKEN*** *: Token SonarQube (masqué)*
- ***TF_VAR_DB_PASSWORD*** *: Mot de passe RDS (masqué)*

.4.4 Détail des Variables CI/CD

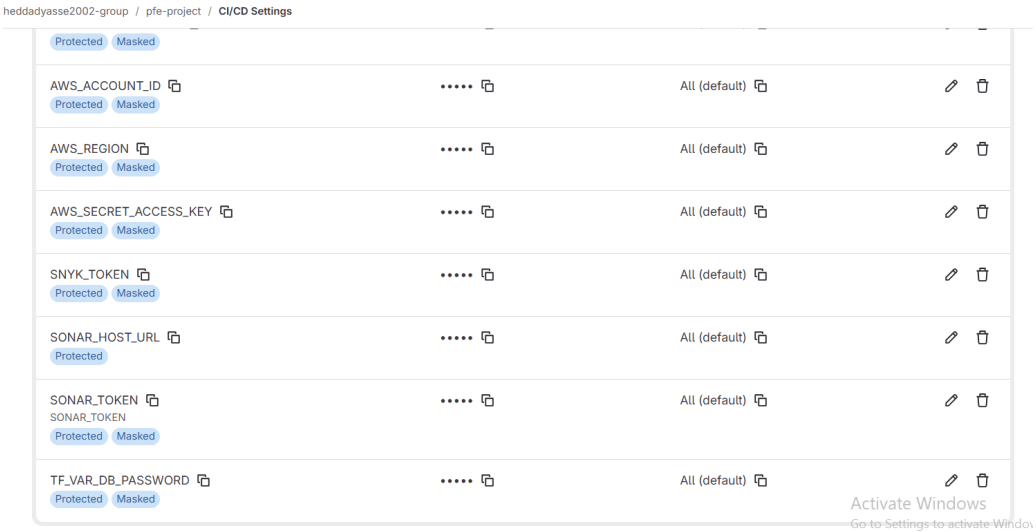


FIGURE 25 – Détail des variables CI/CD avec protection et masquage

Explication : *Les variables CI/CD sont configurées avec des options de sécurité :*

- ***Protected*** *: Variables protégées (uniquement pour les branches protégées)*
- ***Masked*** *: Variables masquées dans les logs (sécurité)*
- ***Environments*** *: Applicables à tous les environnements (All default)*

.4.5 Script SCA (Snyk)

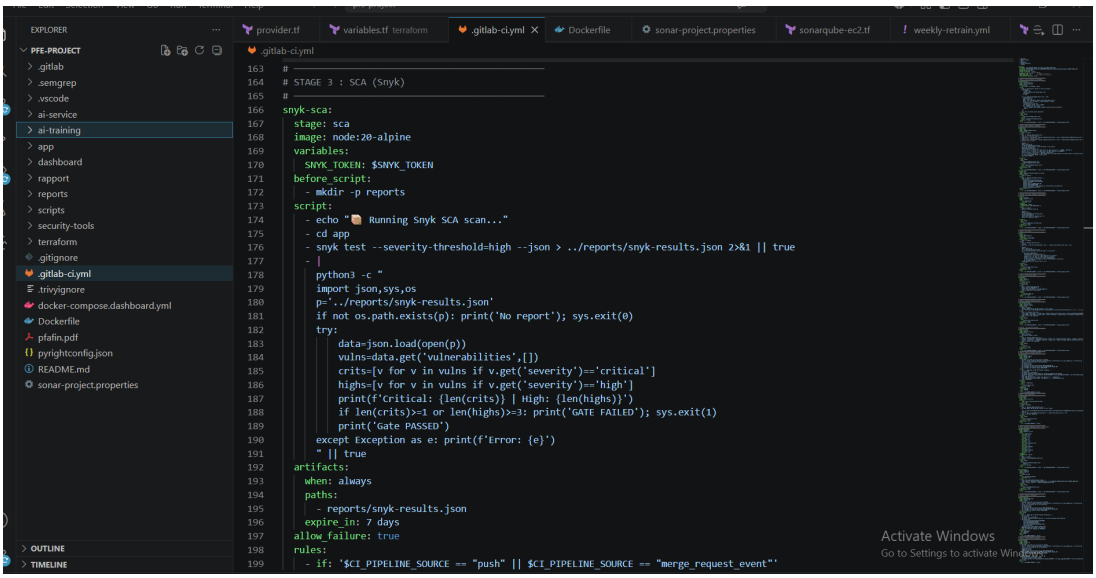


FIGURE 26 – Script SCA - Analyse des dépendances avec Snyk

Explication du script : *Le script Snyk effectue les opérations suivantes :*

1. *Utilise l'image Node.js 20 alpine*
2. *Définit le token Snyk via variable d'environnement*
3. *Exécute* `snyk test` *avec un seuil de sévérité "high"*
4. *Compte les vulnérabilités Critical et High*
5. *Échec si* $\geq$ *1 Critical ou* $\geq$ *3 High*
6. *Sauvegarde du rapport JSON pour 7 jours*
7. *Le job est en mode non-bloquant (*`allow_failure`*)*

.4.6 Exécution du Job de Déploiement

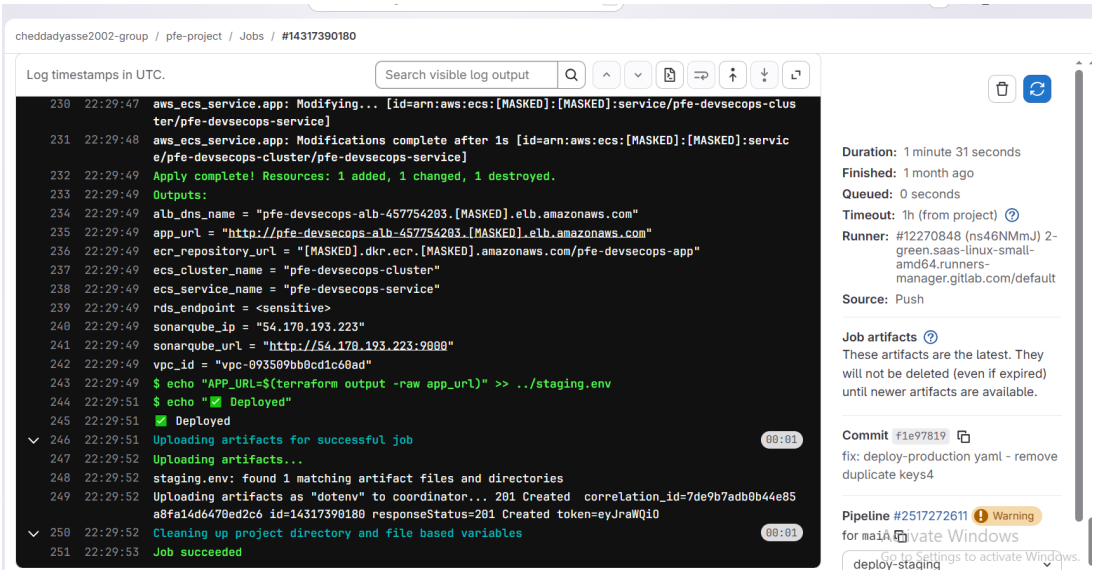


FIGURE 27 – Exécution du job de déploiement Terraform

Explication : *Le job de déploiement Terraform a effectué les opérations suivantes :*

- ***Modifié*** *le service ECS (1 ressource modifiée)*
- ***Ajouté*** *1 ressource*
- ***Détruit*** *1 ressource*
- ***Temps d'exécution*** *: 1 minute 31 secondes*
- ***Outputs*** *: ALB DNS, URL application, ECR, SonarQube IP*

.4.7 Structure du Projet

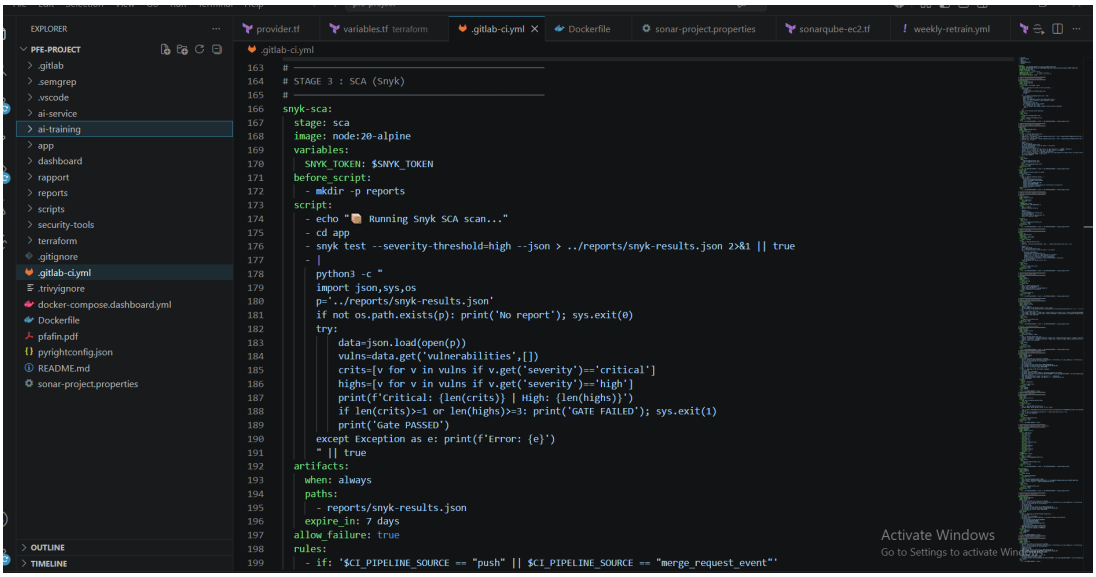


FIGURE 28 – Structure du projet dans GitLab

Explication : *La structure du projet comprend :*

— ***.gitlab-ci.yml*** *: Configuration du pipeline CI/CD*

— ***terraform/*** *: Configuration Terraform (provider.tf, variables.tf)*

— ***ai-service/*** *: Service IA avec FastAPI*

— ***ai-training/*** *: Scripts d'entraînement du modèle*

— ***app/*** *: Application cible (Juice Shop)*

— ***dashboard/*** *: Interface React + FastAPI*

— ***scripts/*** *: Scripts d'automatisation*

— ***security-tools/*** *: Configurations des outils (Semgrep, Checkov)*

— ***reports/*** *: Artefacts du pipeline*